



Database Management Systems

Additional Basic Operations

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL in Database Design

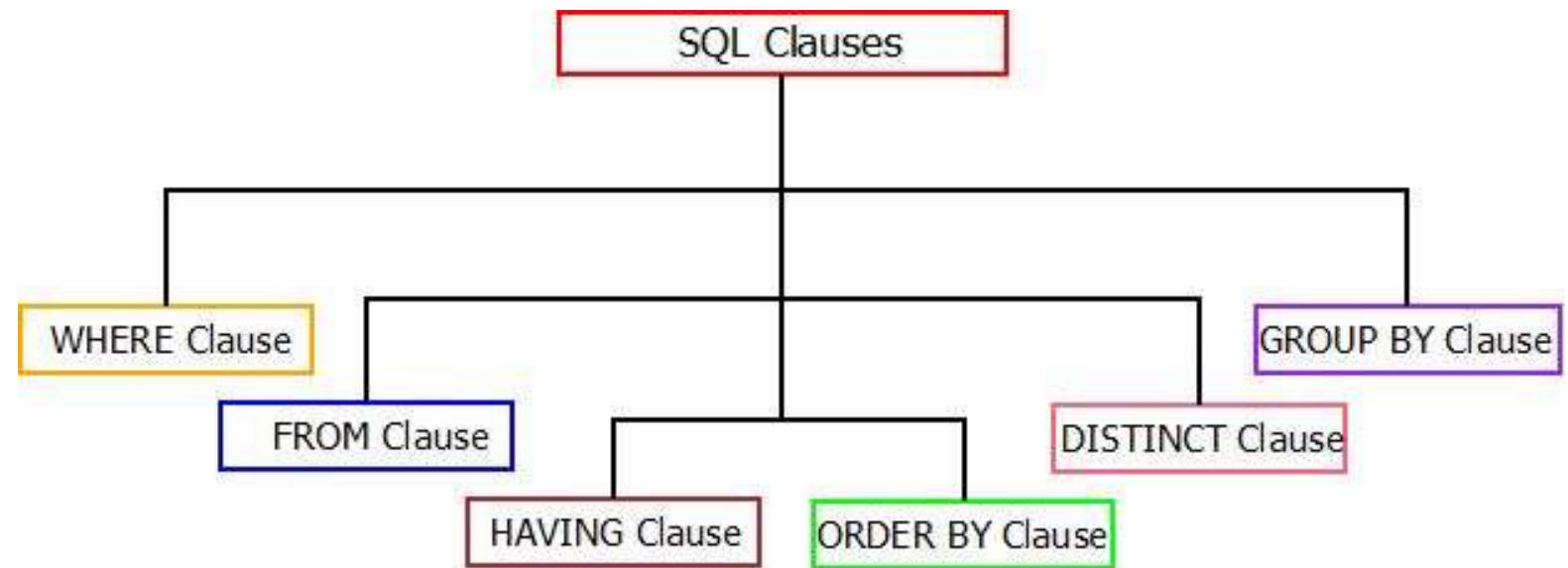
Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Additional Basic Operations

- Select clause
- Distinct clause
- Where clause
- From clause
- Order clause
- Having clause
- Group by clause
- String operations



```
SELECT <attribute and function list>  
FROM <table list>  
[ WHERE <condition> ]  
[ GROUP BY <grouping attribute(s)> ]  
[ HAVING <group condition> ]  
[ ORDER BY <attribute list> ];
```


- The **SELECT** clause lists the attributes desired in the result of a query.
- It corresponds to the projection operation of relational algebra.
- The **FROM** clause is used to specify the table name or the relation name.

Example: Find the names of all employees.

Query: select Fname, Lname from EMPLOYEE;

Output:

Fname	Lname
John	Smith
John	Smith
Franklin	Wong
Joyce	English
Ramesh	Narayan
James	Borg
Jennifer	Wallace
Ahmed	Jabbar
Alicia	Zelaya

NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)

E.g., Name $\equiv$ NAME $\equiv$ name

Some people use upper case wherever we use bold font.

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **DISTINCT** after SELECT.

Example: Retrieve the name and address of all employees (display unique names).

Query: SELECT **distinct** Fname, Lname, Address
FROM EMPLOYEE;

Output:

Fname	Lname	Address
John	Smith	731 Fondren,Houston,TX
Franklin	Wong	638 voss,Houston,TX
Joyce	English	5631 Rice,Houston,TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
James	Borg	450 Stone, Houston,TX
Jennifer	Wallace	291 Berry, Bellaire,Tx
Ahmed	Jabbar	980 Dallas, Houston,TX
Alicia	Zelaya	3321 Castle, Spring,Tx

- The **SELECT** clause can contain arithmetic expressions involving the operations, **+**, **−**, *****, **/** and operating on constants or attributes of tuples.

Query: select fname, salary/2 from EMPLOYEE;

Table:

fname	salary
John	30000.00
John	30000.00
Franklin	40000.00
Joyce	25000.00
Ramesh	38000.00

Output:

fname	salary/2
John	15000.000000
John	15000.000000
Franklin	20000.000000
Joyce	12500.000000
Ramesh	19000.000000

- Query returns a relation that is same as the EMPLOYEE relation, except that the value of the attribute *salary* is divided by 2.
- Can rename “*salary/2*” using the **as** clause:
select *fname*, *salary/2* as *monthly_salary*

- The **WHERE** clause specifies conditions that the result must satisfy (row filter)
- It corresponds to the selection predicate of relational algebra.

Output:

Example: Find all employees in department no 5.

Query: select fname ,dno from EMPLOYEE where dno = 5;

fname	dno
John	5
John	5
Franklin	5
Joyce	5
Ramesh	5

- SQL allows the use of logical connectives **AND**, **OR**, and **NOT**.
- The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.

Output:

- Comparisons can be applied to results of arithmetic expressions

Example: Find all employees in department no 5 with salary lesser than \$70,000.

Query: select fname, salary from employee where dno = 5 and salary < 70000;

fname	salary
John	30000.00
Franklin	40000.00
Joyce	25000.00
Ramesh	38000.00

Example: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

Query: SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Plocation = 'Stafford';

Output:

<u>Pnumber</u>	<u>Dnum</u>	<u>Lname</u>	<u>Address</u>	<u>Bdate</u>
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

- ORDER BY clause is a simple keyword that can be used with SQL aggregate functions, HAVING clause, or GROUP BY clause.
- The main function of the ORDER BY clause is to sort the result in either ascending or descending order.
- It means, sorting the result requires an ORDER BY clause.
- This can be done using either of the two keywords.
 - ASC – For sorting results in ascending order.
 - DESC – For sorting results in descending order.
- If we do not place either “ASC or DESC” at the end of the query, the data is sorted in ascending order by default.

Example1: List the employees based on descending order of their salary.

Query: select fname, lname, salary from EMPLOYEE order by salary desc;

Example2: List the employees based on ascending order of their salary

Query: select fname, lname, salary from EMPLOYEE order by salary asc;

Output1:

fname	lname	salary
James	Borg	55000.00
Jennifer	Wallace	43000.00
Franklin	Wong	40000.00
Ramesh	Narayan	38000.00
John	Smith	30000.00
Joyce	English	25000.00
Ahmed	Jabbar	25000.00
Alicia	Zelaya	25000.00

fname	lname	salary
Joyce	English	25000.00
Ahmed	Jabbar	25000.00
Alicia	Zelaya	25000.00
John	Smith	30000.00
Ramesh	Narayan	38000.00
Franklin	Wong	40000.00
Jennifer	Wallace	43000.00
James	Borg	55000.00

- SQL allows renaming relations and attributes using the **as** clause:

old-name as new-name

Example: Find the annual salary of employees.

Query: select fname,lname, salary*12 as Annual_Income from EMPLOYEE;

Output:

fname	lname	Annual_Income
John	Smith	360000.00
Franklin	Wong	480000.00
Joyce	English	300000.00
Ramesh	Narayan	456000.00
James	Borg	660000.00
Jennifer	Wallace	516000.00
Ahmed	Jabbar	300000.00
Alicia	Zelaya	300000.00

NOTE: Keyword **as** is optional and may be omitted, e.g. instructor **as** T $\equiv$ instructor T

- SQL includes a string-matching operator for comparisons on character strings.
- The operator **LIKE** uses patterns that are described using two special characters:
 - percent (%): The % character matches any substring.
 - underscore (_): The _ character matches any character.

Example: Retrieve all employees whose address is in Houston, Texas.

Query: SELECT Fname, Lname FROM EMPLOYEE WHERE Address LIKE '%Houston,TX%';

Output:

Fname	Lname	Address
John	Smith	731 Fondren,Houston,TX
Franklin	Wong	638 voss,Houston,TX
Joyce	English	5631 Rice,Houston,TX
James	Borg	450 Stone, Houston,TX
Ahmed	Jabbar	980 Dallas, Houston,TX

Note: Match the string “100%” like '100 \%' escape '\' in that above we use backslash (\) as an escape character

Examples: Find all employees who were born during the 1960s.

Query: SELECT Fname, Lname, bdate FROM EMPLOYEE WHERE Bdate LIKE '196____';

Output:

Fname	Lname	bdate
John	Smith	1965-01-09
Ramesh	Narayan	1962-09-15
Ahmed	Jabbar	1969-03-29
Alicia	Zelaya	1968-01-19



Database Management Systems

Set Operations and Null Values

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

SET Operators in SQL



SQL Set operations are used to combine two or more SQL SELECT statements.

Types of Set Operations:

1. Union
2. Union All
3. Intersect
4. Intersect All
5. Except
6. Except All

1. Union

In the UNION operation, the **number of columns and datatypes** must be the same in both the tables on which the UNION operation is being applied.

It **eliminates duplicate rows** from its result set.

Syntax:

```
SELECT column_name FROM table1
```

```
UNION
```

```
SELECT column_name FROM table2;
```

Database Management Systems

Example

Consider the relation: *student_course*(name, course, grade) consisting of records of grades obtained by students in various courses.

The first image displays students and the grades obtained by them in the course 'Physics'

The second image displays students and the grades obtained by them in 'Mathematics'

Further on, these tables will be used to demonstrate the usage of set operations

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Database Management Systems

Union

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
UNION  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

grade
S
A
B

Output :

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

2. Union All

The UNION ALL operation is similar to the UNION operation. It returns the set **without removing duplicates** and sorting the data. If a particular value is present c_1 times in the first set(or relation) and c_2 times in the second set(or relation), the number of duplicates or copies present in the result would be **$c_1 + c_2$** .

Syntax:

```
SELECT column_name FROM table1  
  
UNION ALL  
  
SELECT column_name FROM table2;
```


Database Management Systems

Union All

Query:

Output :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
UNION ALL  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

grade
S
S
A
B
B
S
S
A
A
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Database Management Systems

Union All



Query:

```
SELECT distinct grade FROM student_course  
WHERE course = 'Physics'  
  
UNION ALL  
  
SELECT distinct grade FROM student_course  
WHERE course = 'Mathematics';
```

grade
S
A
B
S
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Notice the difference between the results of the two UNION ALL operations. This is due to the inclusion of the DISTINCT keyword. DISTINCT eliminates any duplicates and hence, the output of each SELECT statement has no duplicates. However, due to the usage of UNION ALL, duplicates are not eliminated in the result of the UNION ALL operation.

3. Intersect

It is used to combine two SELECT statements. The INTERSECT operation returns common rows from both the SELECT statements.

In the Intersect operation, the **number of columns and corresponding data types** must be the same.

It has **no duplicates**.

Syntax:

```
SELECT column_name FROM table1  
  
INTERSECT  
  
SELECT column_name FROM table2;
```

Database Management Systems

Intersect

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
INTERSECT  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

Output :

grade
S
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

4. Intersect All

INTERSECT ALL is similar to the INTERSECT operation. It helps **retain duplicates**. If a particular value is present c_1 times in the first set(or relation) and c_2 times in the second set(or relation), the number of duplicates or copies present in the result would be **$\min(c_1, c_2)$** .

Syntax:

```
SELECT column_name FROM table1  
  
INTERSECT ALL  
  
SELECT column_name FROM table2;
```

Database Management Systems

Intersect All

Query:

```
SELECT grade FROM student_course  
WHERE course = 'Physics'  
  
INTERSECT ALL  
  
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

Output :

grade
S
S
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

5. Except

It combines the result of two SELECT statements. EXCEPT operator is used to display the rows that are present in the first query but absent in the second query.

It has **no duplicates**

Syntax:

```
SELECT column_name FROM table1
```

```
EXCEPT
```

```
SELECT column_name FROM table2;
```

Database Management Systems

Except

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'
```

EXCEPT

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics'
```

EXCEPT

```
SELECT grade FROM student_course  
WHERE course = 'Physics';
```

grade
B

grade

Output :

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

(Empty Set)

6. Except All

EXCEPT ALL is similar to the EXCEPT operation. It helps **retain duplicates**. If a particular value is present c_1 times in the first set(or relation) and c_2 times in the second set(or relation), the number of duplicates or copies present in the result would be **$\max(c_1 - c_2, 0)$** i.e. $c_1 - c_2$ if the difference is positive else 0 (the value itself will not be present)

Syntax:

```
SELECT column_name FROM table1
```

```
EXCEPT ALL
```

```
SELECT column_name FROM table2;
```

Database Management Systems

Except All

Query :

```
SELECT grade FROM student_course  
WHERE course = 'Physics'
```

EXCEPT ALL

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics';
```

```
SELECT grade FROM student_course  
WHERE course = 'Mathematics'
```

EXCEPT ALL

```
SELECT grade FROM student_course  
WHERE course = 'Physics';
```

grade
B
B

grade
A
A

Output :

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Database Management Systems

Null values - Introduction



SQL has various rules for dealing with NULL values.

Generally, NULL has one of the three representations:

1. **Unknown value:** value exists but is not known, or it is not known whether or not the value exists. For example, when a person's date of birth is not known, it is represented as NULL in the database.
2. **Unavailable or withheld value:** value exists but is purposely withheld. For example, a person might not want to disclose his/her date of birth.
3. **Not applicable attributes:** the attribute does not apply to this tuple or is undefined for this tuple. For example, an attribute LastCollegeDegree would be NULL for a person who has no college degrees.

Database Management Systems

Null values - Introduction



- It is often not possible to determine the intended meaning of NULL. Hence, **SQL does not distinguish among the different meanings of NULL.**
- In general, each individual NULL value is considered to be **different** from every other NULL value in the various database records.
- Null values present special problems in relational operations, including arithmetic operations, comparison operations, and set operations.
- Result of an arithmetic operation (+,-,* or /) is **NULL** if **any of the input values are NULL.**

Database Management Systems

Null values in Arithmetic operations

Example:

```
SELECT Hours FROM WORKS_ON;
```

Hours
32.5
7.5
10.0
10.0
10.0
10.0
20.0
20.0
40.0
NULL
15.0
20.0

```
SELECT Hours+10 from WORKS_ON;
```

Hours+10
42.5
17.5
20.0
20.0
20.0
20.0
30.0
30.0
50.0
NULL
25.0
30.0

The first image shows a part of the *Hours* column of the *WORKS_ON* relation. The second image shows the result of adding 10 to each value in the *Hours* column. Notice how the **result of adding 10 to NULL results in NULL.**

- Comparisons involving NULL values pose a greater difficulty.
- To handle comparisons involving NULL values in SQL, a third logical value **UNKNOWN**, in addition to **TRUE** and **FALSE**, is used.
- For example, the expression **(1 < NULL)** would be evaluated as **UNKNOWN**. Similarly, **NOT(1 < NULL)** would be evaluated as **UNKNOWN** as well.
- In other words, any comparison operation involving NULL values would result in an UNKNOWN.

Database Management Systems

Three-valued logic truth table

Truth table for NOT, AND, OR operators:

(a)	AND	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	FALSE	UNKNOWN
	FALSE	FALSE	FALSE	FALSE
	UNKNOWN	UNKNOWN	FALSE	UNKNOWN
(b)	OR	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	TRUE	UNKNOWN	UNKNOWN
(c)	NOT			
	TRUE	FALSE		
	FALSE	TRUE		
	UNKNOWN	UNKNOWN		

Database Management Systems

Is null and is not null



- If the **WHERE** predicate evaluates to **FALSE** or **UNKNOWN** for a tuple, the tuple is not added to the result.
- When a **JOIN** condition is specified, tuples with **NULL** values for the join attributes are not included in the result (unless it is an **OUTER JOIN**)
- SQL uses comparison operators **IS** and **IS NOT** to check if an attribute value is **NULL** or not, respectively

Example: Retrieve the names of all employees who do not have supervisors.

Query: `SELECT Fname, Lname FROM EMPLOYEE WHERE Super_ssn IS NULL;`

Output:

Fname	Lname
James	Borg

Database Management Systems

Is unknown and is not unknown



- SQL allows us to test whether the result of a comparison is UNKNOWN , rather than TRUE or FALSE, by using the clauses **IS UNKNOWN** and **IS NOT UNKNOWN**.

Example: Retrieve the ssn of employees and their corresponding project numbers which satisfy the condition “Hours > 10 is unknown”.

Query: SELECT Essn, Pno FROM WORKS_ON WHERE Hours > 10 IS UNKNOWN;

Output:

Essn	Pno
888665555	20

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
453453453	1	20.0
453453453	2	20.0
666884444	3	40.0
888665555	20	NULL
987654321	20	15.0
987654321	30	20.0

Only those tuples whose *Hours* attribute value is NULL will return UNKNOWN on performing the comparison.

Database Management Systems

Null values in distinct clause



- When a query uses the **SELECT DISTINCT** clause, duplicate tuples are eliminated.
- For this purpose, when comparing values of corresponding attributes from two tuples, the values are treated as **identical** if either **both are non NULL and equal in value**, or **both are NULL**.
- **For example, the tuples ('A', NULL) and ('A', NULL) are treated as identical. Hence, DISTINCT retains only one copy of such distinct tuples.**
- NOTE: Treatment of NULL above is different from the way NULL values are treated in predicates, where a comparison "NULL=NULL" would return UNKNOWN, rather than TRUE.
- The approach of treating tuples as identical if they have the same values for all attributes, even if some of the values are NULL, is also used for the set operations UNION, INTERSECTION, and EXCEPT.

1) When using the LIKE operator for string comparisons, which special character is used to match any single, individual character?

- A) %
- B) *
- C) _
- D) ?

2) Fill in the blank: The _____ operation is used to combine two SELECT statements into a single result set without removing any duplicate rows.

3) According to SQL rules, what is the result if any of the input values in an arithmetic operation (such as +, -, *, or /) is a NULL value?

- A) UNKNOWN
- B) 0
- C) FALSE
- D) NULL

4) Fill in the blank: When using the ORDER BY clause to sort query results, if you do not explicitly place the ASC or DESC keywords at the end of the query, the data will be sorted in _____ order by default.

5) Assume a specific value appears 5 times in Table A and 3 times in Table B. If you execute a query using the EXCEPT ALL operation between Table A and Table B, how many times will that specific value appear in the final result set?

6) Because any direct mathematical comparison involving a NULL value evaluates to UNKNOWN, which specific operator must you use in a WHERE clause to successfully check if an attribute does not have a value?

- A) IS NULL
- B) = NULL
- C) IS UNKNOWN
- D) == NULL

ANSWERS

- 1) C
- 2) UNION ALL
- 3) D
- 4) Ascending
- 5) 2
- 6) A



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Aggregate Functions

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL in Database Design

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary.
- Grouping is used to create subgroups of tuples before summarization.

Built-in aggregate functions:

- **COUNT** - returns the number of tuples or values specified in a query
- **SUM** – returns the sum of a set (or multiset) of numeric values
- **MAX** – returns the maximum value from a set(or multiset) of numeric values
- **MIN** - returns the minimum value from a set(or multiset) of numeric values
- **AVG** – returns the average of a set(or multiset) of numeric values

NOTE: Multiset is a collection similar to a set wherein the multiplicity of every value is greater than or equal to 1. In other words, it is a set that can contain duplicate values.

- The functions MAX and MIN can also be used with attributes that have non-numeric domains if the domain values have a total ordering among one another. DATE, TIME, TIMESTAMP, and alphanumeric strings are examples of such non-numeric domains.
- **Total order** - For any two values in the domain, it can be determined that one appears before the other in the defined order.

Example: Find the sum of salaries of all employees, the maximum salary, the minimum salary, and the average salary

Query: SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary) FROM EMPLOYEE;

Output:

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
281000.00	55000.00	25000.00	35125.000000

- The AS clause can be used to rename column names to more meaningful ones.

Query: SELECT SUM (Salary) AS Total_Sal, MAX (Salary) AS Highest_Sal, MIN (Salary) AS Lowest_Sal, AVG (Salary) AS Average_Sal FROM EMPLOYEE;

Output:

Total_Sal	Highest_Sal	Lowest_Sal	Average_Sal
281000.00	55000.00	25000.00	35125.000000

Example: Retrieve the total number of employees in the company.

Query: SELECT COUNT(*) FROM EMPLOYEE;

Output:

COUNT(*)
8

Example: Find the number of employees in the 'Research' department.

Query: SELECT COUNT(*) FROM EMPLOYEE, DEPARTMENT
WHERE DNO = DNUMBER AND DNAME = 'Research';

Output:

COUNT(*)
4

- In general, COUNT(*) can be used to find the number of rows present in the result of the query.

Example:

Find the sum of salaries of all employees from the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

Query: SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE Dname = 'Research'; //Tables are specified in FROM clause – so join operation

Output:

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
133000.00	40000.00	25000.00	33250.000000

- The COUNT function can be used to count the number of values in a column as well

Example: Count the number of distinct salary values in the database.

Query: SELECT COUNT(DISTINCT Salary) FROM EMPLOYEE;

Output:

COUNT(DISTINCT Salary)
6

- Note the usage of the DISTINCT in the previous query. To understand its significance, we shall run the same query without using DISTINCT.

Query: SELECT COUNT(Salary) FROM EMPLOYEE;

Output:

COUNT(Salary)
8

- SQL, by default, does not eliminate duplicates in the result of a query. Duplicate retention can be specified using the ALL keyword
- If duplicate elimination is required, it must be explicitly specified by using the DISTINCT keyword.
- Notice how there are three copies of the same value in the *Salary* column.
- The total number of values in the *Salary* column equals 8, and hence COUNT (*Salary*) returns 8 (no duplicate elimination).
- By using the DISTINCT keyword, we are specifying duplicate elimination. Hence, out of the three copies, two are discarded and one is retained. Therefore, COUNT(DISTINCT *Salary*) returns 6 (8 – 2 (discarded copies))

Salary
30000.00
40000.00
25000.00
38000.00
55000.00
43000.00
25000.00
25000.00

- In general, when an aggregate function is applied to a collection of values, NULL values are discarded before the calculation.
- **If the collection becomes empty because all the values are NULL, COUNT returns zero and other aggregate functions return NULL.**

Example: Find the average number of hours that employees work on each project.

Query: SELECT Pno, AVG(Hours) as avg_hours FROM WORKS_ON GROUP BY Pno;

Output:

Pno	avg_hours
1	26.25000
2	12.50000
3	25.00000
10	18.33333
20	12.50000
30	18.33333

- Let's have a look at *WORKS_ON* tuples that correspond to *Pno* = 20:

Essn	Pno	Hours
333445555	20	10.0
888665555	20	NULL
987654321	20	15.0

- Notice how one of the entries in the *Hours* column is NULL.
- When AVG is applied to the *Hours* column for *Pno* = 20, it discards the NULL value and considers the other two values. Therefore, the average computation for *Pno* = 20 is as follows:

$$(10 + 15) / 2 = 12.5$$

- Aggregate functions can also be used in selection conditions involving nested queries.
- They can be used in correlated nested queries, which are then placed in the WHERE clause of an outer query.

Example: Retrieve the names of all employees who have two or more dependents.

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE (SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn) >= 2;

Output:

Lname	Fname
Smith	John
Wong	Franklin

The correlated query returns the count of dependents for every employee, which is then checked if it is greater than or equal to 2. All tuples from the *EMPLOYEE* table that satisfy the said condition are then selected.

SQL has two functions that are used alongside comparison operators – **SOME** and **ALL**.

- The **SOME** function **returns True if at least one element in the collection is True.**
- The **ALL** function **returns True if all the elements in the collection are True.**

- In many cases, we would want to apply the aggregate functions to subgroups of tuples in a relation where the subgroups are based on some attribute values.
- For example, we may want to find the average salary of employees in each department or the number of employees who work on each project.
- In such cases, we need to partition the relation into non overlapping subsets(or groups)of tuples.
- The attributes used to partition a relation into groups are called **grouping attributes**. All tuples that have the same values for all grouping attributes are placed in the same group.
- **If NULLs exist in the grouping attribute, then a separate group is created for all tuples with a NULL value in the grouping attribute.**
- Aggregate functions can be applied independently to each group to produce summary information about each group.

- Grouping can be done in SQL using the GROUP BY clause.
- The GROUP BY clause specifies the grouping attributes. They should be present in the SELECT clause as well so that the value of the grouping attributes will appear in the result relation along with the aggregated values.

Example: For each department, retrieve the department number, the number of employees in the department, and their average salary.

Query: SELECT Dno, COUNT(*), AVG (Salary) FROM EMPLOYEE GROUP BY Dno;

Output:

Dno	COUNT(*)	AVG (Salary)
1	1	55000.000000
4	3	31000.000000
5	4	33250.000000

Query: SELECT Dno, COUNT(*), AVG (Salary) FROM EMPLOYEE GROUP BY Dno;

Output:

Dno	COUNT(*)	AVG (Salary)
1	1	55000.000000
4	3	31000.000000
5	4	33250.000000

- In the above query, *Dno* is the grouping attribute. The *EMPLOYEE* relation is partitioned into groups in such a way that each group has tuples with the same value of *Dno*.
- Hence, each group consists of employees who work for the same department.
- The COUNT and AVG functions are applied to each group independently and are displayed in the result relation.
- The SELECT clause must contain only the grouping attributes and aggregate functions applied on each group of tuples.

An illustration of how grouping works is provided for the previous example

Fname	Minit	Lname	<u>Ssn</u>	...	Salary	Super_ssn	Dno
John	B	Smith	123456789	...	30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453		25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

Grouping EMPLOYEE tuples by the value of Dno

Example: For each project, retrieve the project number, the project name, and the number of employees who work on that project.

Query: SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno GROUP BY Pnumber, Pname;

Output:

Pnumber	Pname	COUNT(*)
10	Computerization	3
30	Newbenefits	3
1	ProductX	2
2	ProductY	3
3	ProductZ	2
20	Reorganization	3

- The above query is an example of using GROUP BY along with JOIN.
- In such statements, relations are first joined using the condition specified in the WHERE clause. Grouping and aggregation are then performed on the result of this joining

- There are some scenarios wherein we would want to retrieve the values of aggregate functions for groups that satisfy certain conditions.
- SQL provides the HAVING clause, which can appear in conjunction with a GROUP BY clause, for this purpose.
- **The HAVING clause is used to specify conditions for groups of tuples, whereas the WHERE clause is used to specify conditions for individual tuples.**
- HAVING provides a condition on the summary information regarding the group of tuples associated with each value of the grouping attributes.
- **Only the groups that satisfy the condition are retrieved in the result of the query.**

Example:

For each project on which more than two employees work, retrieve the project number, the project name, and the number of employees who work on the project.

Query:

```
SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON  
WHERE Pnumber = Pno GROUP BY Pnumber, Pname HAVING COUNT(*) > 2;
```

Output:

Pnumber	Pname	COUNT(*)
10	Computerization	3
30	Newbenefits	3
2	ProductY	3
20	Reorganization	3

An illustration of how the HAVING clause works is provided for the previous example

Pname	Pnumber	...	Essn	Pno	Hours
ProductX	1		123456789	1	32.5
ProductX	1		453453453	1	20.0
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
ProductZ	3		666884444	3	40.0
ProductZ	3		333445555	3	10.0
Computerization	10	...	333445555	10	10.0
Computerization	10		999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

These groups are not selected by the HAVING condition of Q26.

After applying the WHERE clause but before applying HAVING

Pname	Pnumber	...	Essn	Pno	Hours
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
Computerization	10		333445555	10	10.0
Computerization	10	...	999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

Pname	Count (*)
ProductY	3
Computerization	3
Reorganization	3
Newbenefits	3

Result of Q26
(Pnumber not shown)

After applying the HAVING clause condition

Example: For each project, retrieve the project number, the project name, and the number of employees from department 5 who work on that project.

Query: SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Dno = 5 AND Ssn=Essn
GROUP BY Pnumber, Pname;

Output:

Pnumber	Pname	COUNT(*)
1	ProductX	2
2	ProductY	3
3	ProductZ	2
10	Computerization	1
20	Reorganization	1

- We must be extra careful when two different conditions apply – one specified in the WHERE clause and the other specified in the HAVING clause.
- **The rule is that the condition specified in the WHERE clause is executed first, to select the individual tuples, and then the HAVING clause is executed, to select groups of tuples.**
- **This means that the HAVING clause groups only those tuples that are selected by the WHERE clause.**
- Consider the following example:

Retrieve the count of number of employees whose salary exceeds \$40,000 in each department, but only for those departments where more than two employees work.

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE

WHERE Salary>40000 GROUP BY Dno HAVING COUNT(*) > 2;

- At the outlook, this might seem like the correct query for the given question. However, closer inspection reveals that this might not be the case.
- As stated earlier, the WHERE clause is executed first and its result is what the HAVING clause works on.
- In other words, the WHERE clause first returns only those employees whose salary exceeds \$40,000. This is followed by the grouping of the filtered relation and the selection of groups by the HAVING clause condition.
- Hence, only departments with more than two employees who each earn more than \$40,000 are selected, which is not the expected result.

Database Management Systems

Where predicate and Having predicate



The correct query is given below

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN
(SELECT Dno FROM EMPLOYEE GROUP BY Dno HAVING COUNT(*) > 2)
GROUP BY Dno;

Output:

Dno	COUNT(*)
4	1

This shows that there is only one employee with a salary greater than \$40,000 belonging to a department(here department 4) with more than two employees. All other departments either have two or fewer employees or don't have employees whose salary exceed \$40,000.

Let's try executing the incorrect query

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE

WHERE Salary>40000 GROUP BY Dno HAVING COUNT(*) > 2;

Output:

Dno	COUNT(*)
-----	----------

- The output of the query is an empty relation. This is because the query returns only those departments with more than two employees whose salaries exceed \$40,000.
- From the output of the correct query, we can see that there are no departments that satisfy this condition. Hence, an empty relation is returned.

Aspect	WHERE	HAVING
Purpose	Filters rows before grouping/aggregation	Filters groups/aggregated results after GROUP BY
When is it Applied	Applied before grouping	Applied after grouping
Works On	Works on individual row values (columns)	Works on aggregated values (like sum(), count() etc)
Use of Aggregates	Cannot use aggregate functions.	Can use aggregate functions.
Order of execution	First WHERE is executed then subsequent statements	HAVING is executed after WHERE has been applied

1) Which of the following built-in aggregate functions can be used with non-numeric domains (such as DATE, TIME, and alphanumeric strings) provided there is a total ordering among the values?

- A) SUM and AVG
- B) COUNT and SUM
- C) MAX and MIN
- D) AVG and MIN

2) If a collection of values becomes empty because all the values are NULL, the COUNT function returns zero, but all other aggregate functions return _____.

3) Fill in the blank: While the WHERE clause is used to specify conditions for individual tuples, the HAVING clause is exclusively used to specify conditions for _____ of tuples.

4) When using the GROUP BY clause, what happens if NULL values exist within the specified grouping attribute?

- A) The query throws a syntax error.
- B) The tuples with NULL values are completely discarded from the result.
- C) A separate group is created for all tuples with a NULL value in the grouping attribute.
- D) The NULL values are randomly assigned to other existing groups.

5) When a single SQL query contains both a WHERE clause and a HAVING clause, what is the correct order of execution?

- A) The HAVING clause is executed first to filter groups, then WHERE filters the individual rows.
- B) The WHERE clause is executed first to select individual tuples, and then the HAVING clause is executed to select groups of tuples.
- C) They are executed simultaneously.
- D) The execution order depends on the aggregate functions used in the SELECT clause.

6) When using functions alongside comparison operators in nested queries, which function returns True if all the elements in the collection evaluate to True?

- A) SOME
- B) EXISTS
- C) ANY
- D) ALL

Answers:

1. C
2. NULL
3. groups (or subgroups)
4. C
5. B
6. D



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Join expressions

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL in Database Design

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- Join operations take two relations and return as a result another relation.
- A join operation is a Cartesian product that requires tuples in the two relations to match (under some conditions).
- It also specifies the attributes that are present in the result of the join
- The join operations are typically used as subquery expressions in the **FROM** clause

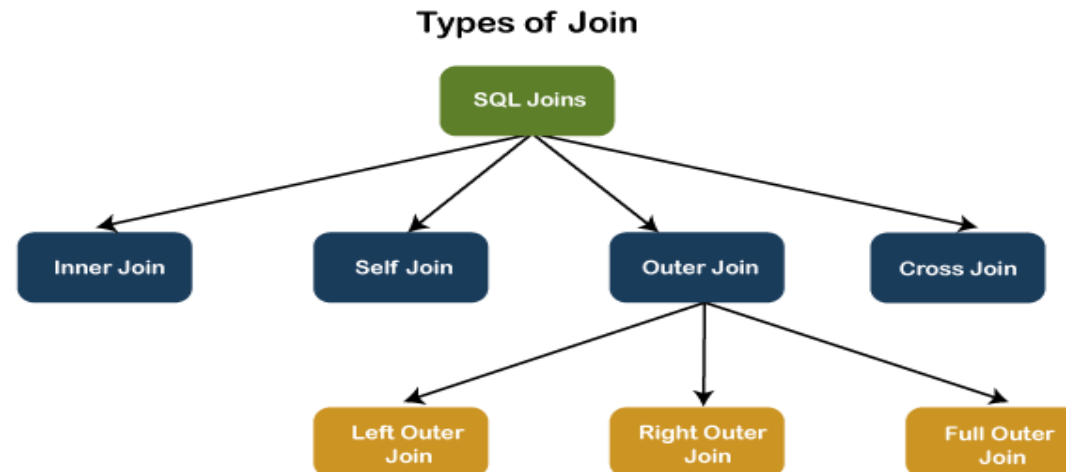
Three types of joins:

Natural join

Inner join

Outer join

Cross join



- **A natural join is a type of join operation that creates an implicit join by combining tables based on columns with the same name and data type.**
- In a NATURAL JOIN on two relations R and S, no join condition is specified.
- An implicit EQUIJOIN condition for each pair of attributes with the same name from R and S is created.
- Each such pair of attributes is included only once in the resulting relation.
- If the names of the join attributes are not the same in the base relations, it is possible to rename the attributes so that they match, and then to apply NATURAL JOIN.

Points to remember:

- There is no need to specify the column names for natural join.
- The resultant table always contains unique columns.
- It is possible to perform a natural join on more than two tables.
- Do not use the ON clause.

The following is a basic syntax to illustrate the natural join:

```
SELECT [column_names | *]  
FROM table_name1  
NATURAL JOIN table_name2;
```

Database Management Systems

Natural Join



Employee Table:

Employee_ID	employee_name	employee_salary
1	Arun Tiwari	50000
2	Sachin Rathi	64000
3	Harshal Pathak	48000
4	Arjun Kuwar	46000
5	Sarthak Gada	62000
6	Saurabh Sheik	53000
7	Shubham Singh	29000
8	Shivam Dixit	54000
9	Vicky Gujral	39000
10	Vijay Bose	28000
NULL	NULL	NULL

Department	Department_name	Employee_ID
1	Production	1
2	Sales	3
3	Marketing	4
4	Accounts	5
5	Development	7
6	HR	9
7	Sales	10
NULL	NULL	NULL

Loan:

loan_id	branch	amount
1	B1	15000
2	B2	10000
3	B3	20000
4	B4	100000
5	B5	150000
6	B6	50000
7	B7	35000
8	B8	85000
NULL	NULL	NULL

CustID	CustName	loan_id
1	Sonakshi Dixit	1
2	Shital Garg	4
3	Swara Joshi	5
4	Isha Deshmukh	2
5	Swati Bose	7
6	Asha Kapoor	10
7	Nandini Shah	9
NULL	NULL	NULL

Example: List the employees along with their department names.

Query: SELECT employee_name, employee_salary, Department_name
FROM employee NATURAL JOIN Department;

Output:

employee_name	employee_salary	Department_name
Arun Tiwari	50000	Production
Harshal Pathak	48000	Sales
Arjun Kuwar	46000	Marketing
Sarthak Gada	62000	Accounts
Shubham Singh	29000	Development
Vicky Gujral	39000	HR
Vijay Bose	28000	Sales

Example: List the customers along with their loan amount and address.

Query: SELECT CustName, CustID, amount, mobile ,address
FROM Borrower NATURAL JOIN loan NATURAL JOIN cust_info ;

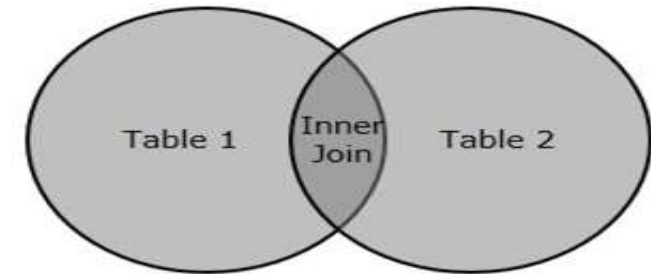
Output:

CustName	CustID	amount	mobile	address
Sonakshi Dixit	1	15000	598675498654	777 Brockton Avenue, Abington MA 251
Shital Garg	2	100000	698853747888	337 Russell St, Hadley MA 103
Swara Joshi	3	150000	234456977555	20 Soojian Dr, Leicester MA 154
Isha Deshmukh	4	10000	987656789666	780 Lynnway, Lynn MA 19
Swati Bose	5	35000	756489372222	700 Oak Street, Brockton MA 23

Customer info table:

CustID	mobile	address
1	598675498654	777 Brockton Avenue, Abington MA 251
2	698853747888	337 Russell St, Hadley MA 103
3	234456977555	20 Soojian Dr, Leicester MA 154
4	987656789666	780 Lynnway, Lynn MA 19
5	756489372222	700 Oak Street, Brockton MA 23

- SQL Inner Join is a type of join that is used to combine records from two related tables, based and select the rows depending on the condition (join predicate)
- Inner Join query compares each row of the first table with each row of the second table to find all pairs of rows that satisfy the join-predicate.
- When the join-predicate is satisfied, column values for each matched pair of rows of both tables are combined into a result row.
- The Inner Join is a default join; i.e., even if the “Join” keyword is used instead of “Inner Join”, tables are joined using matching records of common columns, by default.
- The most important and frequently used join in SQL is the Inner Join. It is also referred as Equijoin.



Example : Write a query to perform inner join considering employee table and department table.

Query:

```
SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID,  
d.Department_Name FROM employee e INNER JOIN department d ON e.Employee_ID =  
d.Employee_ID;
```

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

Example Write a query to perform inner join considering loan table and borrower table.

Query:

```
SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l JOIN  
Borrower b ON l.Loan_ID = b.Loan_ID;
```

Output:

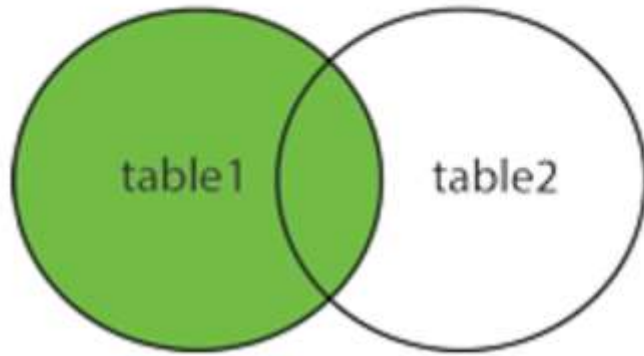
Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
2	B2	10000	4	Isha Deshmukh
7	B7	35000	5	Swati Bose

- An extension of the join operation that avoids loss of information in few cases.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- In the SQL outer JOIN, all the content from both the tables is integrated together.
- Even though the records from both the tables are matched or not, the matching and non-matching records from both the tables will be considered an output of the outer join in SQL.

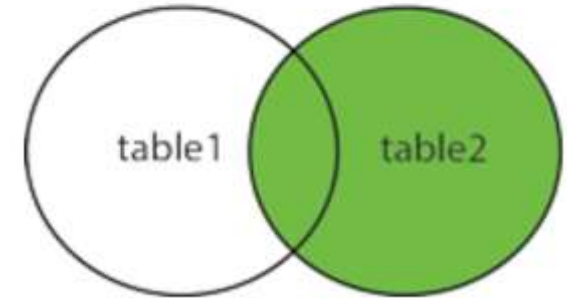
There are three different types of outer join in SQL:

Left Outer Join
Right Outer Join
Full Outer Join

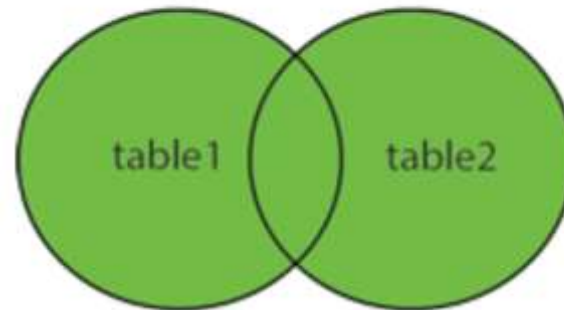
LEFT JOIN



RIGHT JOIN



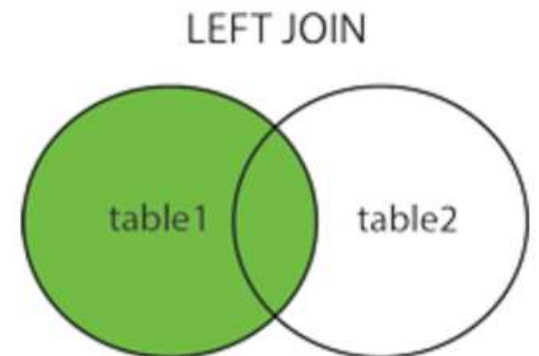
FULL OUTER JOIN



- In LEFT OUTER JOIN every tuple in the left table must appear in the result. If it does not have a matching tuple, it is padded with NULL values for the attributes of the right table.
- If we use the left outer join to combine two different tables, then we will get all the records from the left table.
- But we will get only those records from the right table, which have the corresponding key in the left table ie the matching tuple

Syntax:

```
SELECT TableName1.columnName1, TableName2.columnName2  
FROM TableName1 LEFT OUTER JOIN TableName2 ON  
TableName1.ColumnName = TableName2.ColumnName;
```



Example : Write a query to perform left outer join considering employee table as the left table and department table as the right table.

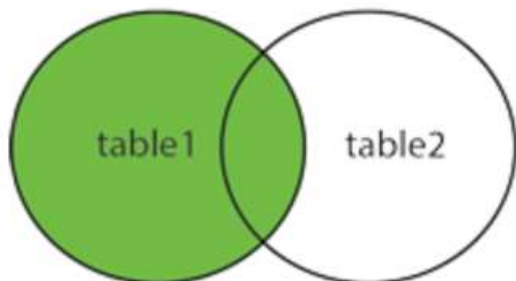
Query:

```
SELECT    e.Employee_ID,    e.Employee_Name,    e.Employee_Salary,    d.DepartmentID,  
d.Department_Name FROM employee e LEFT OUTER JOIN department d ON e.Employee_ID =  
d.Employee_ID;
```

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
2	Sachin Rathi	64000	NULL	NULL
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
6	Saurabh Sheik	53000	NULL	NULL
7	Shubham Singh	29000	5	Development
8	Shivam Dixit	54000	NULL	NULL
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

LEFT JOIN

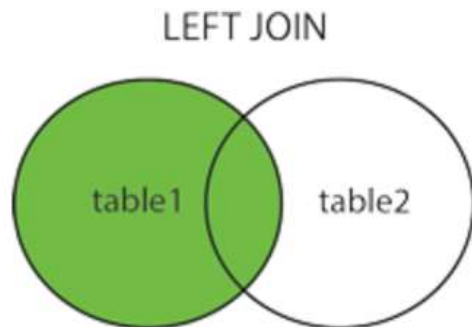


Example Write a query to perform left outer join considering loan table as the left table and borrower table as the right table.

Query:

```
SELECT L.Loan_ID, L.Branch, L.Amount, B.CustID, B.CustName FROM Loan L LEFT  
OUTER JOIN Borrower B ON L.Loan_ID = B.Loan_ID;
```

Output:

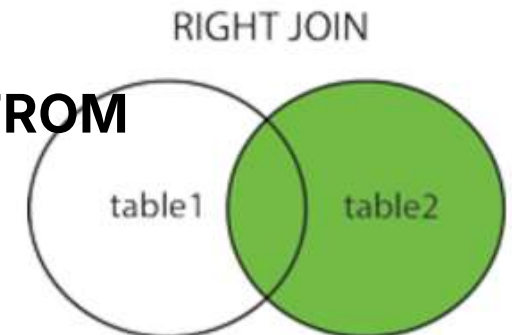


Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
2	B2	10000	4	Isha Deshmukh
3	B3	20000	NULL	NULL
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
6	B6	50000	NULL	NULL
7	B7	35000	5	Swati Bose
8	B8	85000	NULL	NULL

- In RIGHT OUTER JOIN every tuple in the right table must appear in the result. If it does not have a matching tuple, it is padded with NULL values for the attributes of the left table. Right outer join is the reverse of the left outer join.
- If we use the right outer join to combine two different tables, then we will get all the records from the right table.
- But we will get only those records from the left table, which have the corresponding key in the right table.

Syntax :

```
SELECT TableName1.columnName1, TableName2.columnName2 FROM  
TableName1 RIGHT OUTER JOIN TableName2 ON  
TableName1.ColumnName = TableName2.ColumnName;
```

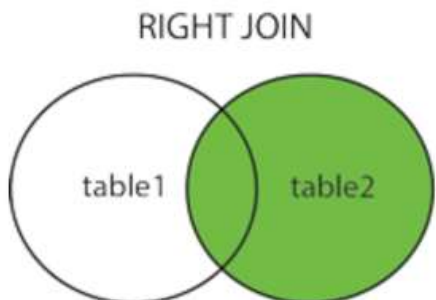


Example:

Write a query to perform right outer join considering employee table as the left table and department table as the right table.

Query:

```
SELECT    e.Employee_ID,    e.Employee_Name,    e.Employee_Salary,    d.DepartmentID,  
d.Department_Name FROM employee e RIGHT OUTER JOIN department d ON e.Employee_ID =  
d.Employee_ID;
```



Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

Example:

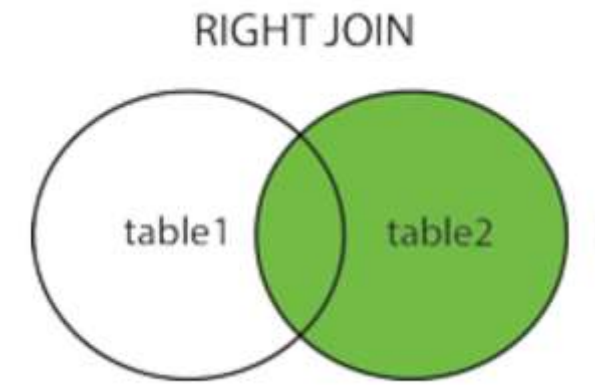
Write a query to perform right outer join considering the loan table as the left table and the borrower table as the right table.

Query:

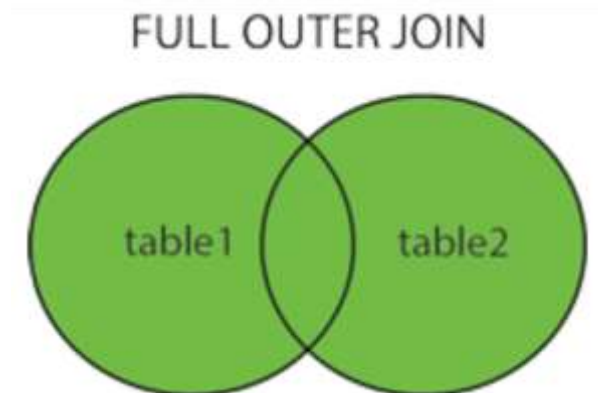
```
SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName
FROM Loan l RIGHT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID;
```

Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
2	B2	10000	4	Isha Deshmukh
7	B7	35000	5	Swati Bose
NULL	NULL	NULL	6	Asha Kapoor
NULL	NULL	NULL	7	Nandini Shah



- If we use a full outer join to combine two different tables, then we will get all the records from both tables.
- We will get all the records from the left table as well as the right table.
- **MySQL doesn't support FULL OUTER JOIN directly.**
- So to implement full outer join in MySQL, we will execute two queries in a single query.
- The first query will be of LEFT OUTER JOIN, and the second query will be of RIGHT OUTER JOIN.
- We will combine the first and the second query with the UNION operator to see the results of FULL OUTER JOIN.



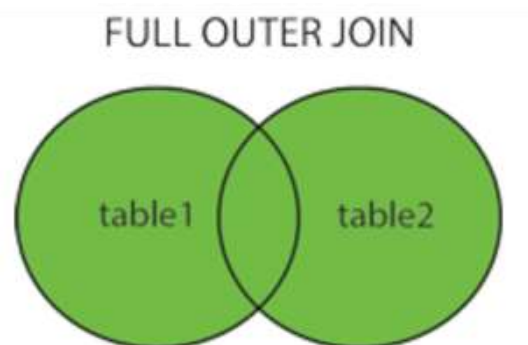
Example 1:

Write a query to perform full outer join considering the employee table as the left table and department table as the right table.

Query: `SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e LEFT OUTER JOIN department d ON e.Employee_ID = d.Employee_ID UNION SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e RIGHT OUTER JOIN department d ON e.Employee_ID = d.Employee_ID;`

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales
2	Sachin Rathi	64000	NULL	NULL
6	Saurabh Sheik	53000	NULL	NULL
8	Shivam Dixit	54000	NULL	NULL



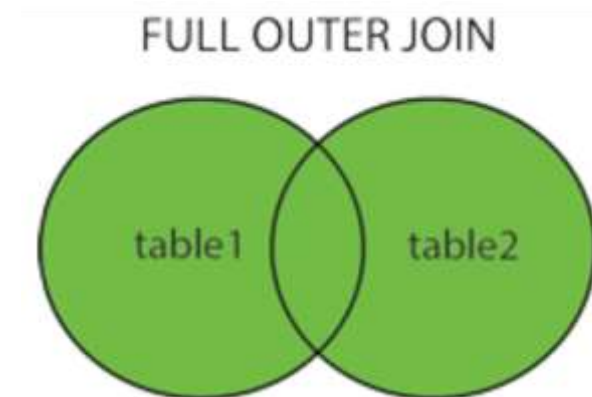
Example 1:

Write a query to perform full outer join considering loan table as the left table and borrower table as the right table.

Query: SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l LEFT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID UNION SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l RIGHT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID;

Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
2	B2	10000	4	Isha Deshmukh
3	B3	20000	NULL	NULL
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
6	B6	50000	NULL	NULL
7	B7	35000	5	Swati Bose
8	B8	85000	NULL	NULL
NULL	NULL	NULL	6	Asha Kapoor
NULL	NULL	NULL	7	Nandini Shah



- Cross Join returns the Cartesian Product
- It does not use a join condition (ON ...), so it doesn't care about matching columns.
- **For example**, consider the tables loan (having 8 rows) and borrower (having 7 rows)
- Rows Returned: (Rows in Table A) * (Rows in Table B) = $8 \times 7 = 56$
- **Query: select * from loan cross join borrower;**

```
mysql> select *  
      -> from loan  
      -> cross join borrower;  
+-----+-----+-----+-----+-----+-----+  
| loan_id | branch | amount | CustID | CustName      | loan_id |  
+-----+-----+-----+-----+-----+-----+  
|        1 | B1     | 15000  |        7 | Nandini Shah  |        9 |  
|        1 | B1     | 15000  |        6 | Asha Kapoor   |       10 |  
+-----+-----+-----+-----+-----+-----+  
|        8 | B8     | 85000  |        4 | Isha Deshmukh |        2 |  
|        8 | B8     | 85000  |        3 | Swara Joshi    |        5 |  
|        8 | B8     | 85000  |        2 | Shital Garg    |        4 |  
|        8 | B8     | 85000  |        1 | Sonakshi Dixit |        1 |  
+-----+-----+-----+-----+-----+-----+  
56 rows in set (0.00 sec)
```

1) Which of the following statements is true regarding a NATURAL JOIN?

- A) It requires an explicit ON clause to match columns.
- B) Each pair of matching attributes is included twice in the resulting relation.
- C) It creates an implicit EQUIJOIN condition for each pair of attributes with the same name and data type.
- D) It pads non-matching records with NULL values.

2) Fill in the blank: The INNER JOIN is the default join in SQL, meaning if you just use the "Join" keyword, tables are joined using matching records of common columns by default. It is also commonly referred to as an _____.

3) When performing a LEFT OUTER JOIN, what happens to a tuple in the left table if it does not have a matching tuple in the right table?

- A) It is completely omitted from the result.
- B) It causes the query to return an empty set.
- C) It is padded with NULL values for the attributes of the right table.
- D) It drops the unmatched columns from the final output.

4) Fill in the blank: Because MySQL does not support the FULL OUTER JOIN keyword directly, it must be implemented by executing a LEFT OUTER JOIN and a RIGHT OUTER JOIN and combining them using the _____ operator.

5) A CROSS JOIN returns the Cartesian Product of two tables. If a loan table has 8 rows and a borrower table has 7 rows, how many rows will be returned by executing a CROSS JOIN between them?

- A) 8
- B) 15
- C) 56
- D) 0

6) Fill in the blank: Unlike inner and outer joins, a CROSS JOIN does not care about matching columns because it does not use a _____ condition (such as the ON clause).

Answers

1. C
2. Equijoin
3. C
4. UNION
5. C
6. join



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Nested Queries

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A **nested query** can be defined as a **complete SELECT-FROM-WHERE block** within another SQL query. The other query is called the **outer query**.
- Nested queries are used to fetch existing values in the database that can be used by the outer query for comparison.
- **They can appear in the WHERE clause, FROM clause, SELECT clause, and other SQL clauses as needed.**
- SQL provides a comparison operator **IN**, which checks if a value is present in a set(or multiset) of values. **It returns True if the value is present in the set and False** otherwise. It essentially checks for Set membership.
- The **NOT IN** operator returns True if the value is not present in the set and False otherwise.

Example: Retrieve the list of project numbers of projects that have an employee with the last name 'Smith' involved either as a manager or as a worker.

Query: SELECT DISTINCT Pnumber
FROM PROJECT WHERE Pnumber IN
(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')
OR Pnumber IN
(SELECT Pno FROM WORKS_ON, EMPLOYEE
WHERE Essn = Ssn AND Lname = 'Smith');

Output:

Pnumber
1
2

- Let's understand how this query works.
- Consider the first sub-query:

**(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')**

- This query selects the project number of all projects handled by departments whose manager's last name is 'Smith'.
- On executing this subquery, we obtain an empty set. The output is shown below:

Pnumber (Empty Set)

- This is because there is no manager whose last name is 'Smith'.

Manager_last_name
Wong
Borg
Wallace

Consider the second subquery:

```
( SELECT Pno FROM WORKS_ON, EMPLOYEE  
  WHERE Essn = Ssn AND Lname = 'Smith' );
```

- This subquery selects the project number of all projects that employees with the last name 'Smith' work on.
- The output of this subquery is shown below:

Pno
1
2

(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith') → ()

(SELECT Pno FROM WORKS_ON, EMPLOYEE
WHERE Essn = Ssn AND Lname = 'Smith'); → (1, 2)

By plugging in the results of the nested queries in the outer query, we obtain the equivalent query:

SELECT DISTINCT Pnumber
FROM PROJECT WHERE Pnumber IN
()
OR Pnumber IN
(1, 2)

- In general, a nested query **returns a table(relation)** which is a set(or multiset) of tuples.
- However, there are certain scenarios wherein a nested query returns a table with **a single attribute and a single tuple**.
- In such cases, we can treat the result as a **scalar** i.e. a single value, and apply operations applicable to singular values.
- For example, the equal operator (=) is not applicable when one of the operands is a set of values(which is usually what a nested query returns).
- **However, when a nested query returns a single value, we can use the = operator instead of the IN operator.**
- SQL allows the use of tuples of values in comparisons by placing them within parentheses.

Example: Retrieve the ssn of all employees who work on any of the same (project, hours) combination that employee 'John Smith' (ssn = '123456789') works on.

Query: `SELECT DISTINCT Essn FROM WORKS_ON
WHERE (Pno, Hours) IN
(SELECT Pno, Hours FROM WORKS_ON
WHERE Essn = '123456789');`

Output:

Essn
123456789

- The IN operator compares the subtuple of values in parentheses (Pno, Hours) within each tuple in *WORKS_ON* with the set of type-compatible tuples produced by the nested query.

- In addition to the IN and NOT IN operators, a number of other comparison operators can be used to compare a single value to a set or multiset of values.
- These operators are of the form – ***comp_op* [ANY | ALL | SOME]**
- ***comp_op*** denotes comparison operators such as =, <> (not equals), >, >=, <, <= which are used to compare two values.
- ***comp_op ALL*** returns True if the result of applying ***comp_op*** between the single value and an element of the set is True **for every element of the set**. For example, the expression $1 < ALL(2,3)$ evaluates as True because 1(the single value here) is lesser than(comp_op) both 2 and 3(the elements of the set operand).
- ***comp_op SOME(or ANY)*** returns True if the result of applying ***comp_op*** between the single value and an element of the set is True **for at least one element in the set**.

Example: Retrieve the last and first names of all employees whose salary is greater than the salary of all the employees in department 5

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);

Output:

Lname	Fname
Borg	James
Wallace	Jennifer

- First, the nested query gets executed. It returns the salary of all employees who work in department 5. The output is shown below:

Salary
30000.00
40000.00
25000.00
38000.00

- The outer query essentially becomes:

```
SELECT Lname, Fname FROM EMPLOYEE
```

```
WHERE Salary > ALL (30000.00, 40000.00, 25000.00, 38000.00)
```

- For every tuple in the *EMPLOYEE* relation, the value in the *salary* attribute is compared with the output of the nested query using > ALL. The WHERE clause evaluates to True if the value in the *salary* column for that particular tuple is greater than all the values in the nested query output. A table with the last name, first name, and salary of employees is shown to verify the result.
- The same can be achieved using the MAX function.**

Lname	Fname	Salary
Smith	John	30000.00
Wong	Franklin	40000.00
English	Joyce	25000.00
Narayan	Ramesh	38000.00
Borg	James	55000.00
Wallace	Jennifer	43000.00
Jabbar	Ahmed	25000.00
Zelaya	Alicia	25000.00

Example: Retrieve the last and first names of all employees whose salary is greater than the salary of at least one employee in department 5

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE Salary > SOME (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);

Output:

Lname	Fname
Smith	John
Wong	Franklin
Narayan	Ramesh
Borg	James
Wallace	Jennifer

Notice how there are more employees who satisfy the given condition as compared to the previous example. This is because > SOME(and > ANY) evaluates as True if the employee salary in the outer query is greater than even one value in the nested query output.

- The previous example could be solved using the MIN function as well.
- = SOME or = ANY is equivalent to the IN operator.
- <> ALL is equivalent to the NOT IN operator.

- In general, we can have several levels of nested queries.
- **Ambiguity among attribute names arises if attributes belonging to the relation in the FROM clause of the outer query and the relation in the FROM clause of the nested query have the same name.**
- The rule is that a **reference to an unqualified attribute refers to the relation declared in the innermost query**

For example, consider the given query for the example :

“Retrieve the project numbers of all projects either managed by an employee named 'Smith' or worked on by an employee named 'Smith'.”

The relation PROJECT has been referenced both in the outer query and the inner query. The attributes Pnumber and Dnum in the nested query correspond to the PROJECT relation in the nested query and not the one in the outer query.

```
SELECT      DISTINCT Pnumber
FROM        PROJECT
WHERE       Pnumber IN
            ( SELECT      Pnumber
              FROM        PROJECT, DEPARTMENT, EMPLOYEE
              WHERE       Dnum = Dnumber AND
                          Mgr_ssn = Ssn AND Lname = 'Smith' )

OR

Pnumber IN
( SELECT      Pno
  FROM        WORKS_ON, EMPLOYEE
  WHERE       Essn = Ssn AND Lname = 'Smith' );
```

To refer to an attribute of a relation specified in the outer query, we specify and refer to an alias for that relation.

Consider the following query for the example :

“Retrieve the first and last names of employees who have a dependent with the same name and same sex as themselves.”

```
SELECT      E.Fname, E.Lname
FROM        EMPLOYEE AS E
WHERE       E.Ssn IN ( SELECT      D.Essn
                        FROM        DEPENDENT AS D
                        WHERE       E.Fname = D.Dependent_name
                        AND E.Sex = D.Sex );
```

- Here E is the alias for the relation *EMPLOYEE*. There are two attributes named Sex – one in the *EMPLOYEE* table and another in the *DEPENDENT* table.
- We must qualify E.Sex because it refers to the Sex attribute of the *EMPLOYEE* relation from the outer query, and not to the Sex attribute of the *DEPENDENT* relation.

- If there were any unqualified references to **Sex** in the nested query, they would refer to the **Sex** attribute of the *DEPENDENT* relation.
- However, we would not have to qualify attributes *such as Ssn* (of the *EMPLOYEE* relation) if they appeared in the nested query because the *DEPENDENT* relation does not have an attribute called *Ssn*, so there is no ambiguity.
- It is generally advisable to create tuple variables (aliases) for all the tables referenced in an SQL query to avoid potential errors and ambiguities

- SQL allows a subquery expression to be used in the FROM clause.
- Since a subquery returns a relation as a result, it can be inserted in a SELECT-FROM-WHERE expression at any place where a relation can appear.

Example: Find the average employee salary of those departments where the average salary is greater than \$32,000.

- One way to frame the query would be to use the GROUP BY and HAVING clauses.
- We can achieve the same by using nested subqueries. First, extract department numbers and their corresponding average salaries from the *EMPLOYEE* table using the following query:

```
SELECT Dno, AVG(Salary) FROM EMPLOYEE GROUP BY Dno
```

- The resulting relation can be then be filtered using the WHERE clause.

Query: SELECT Dno, ROUND(AVG(Salary),2) as avg_salary
FROM EMPLOYEE
GROUP BY Dno
HAVING AVG(Salary) > 32000;

Output :

Dno	avg_salary
1	55000.00
5	33250.00

NOTE: The **ROUND()** function in MySQL is used to round a number to a specified number of decimal places. Its syntax is given by ROUND(X, D) where X is the number to be rounded and D (optional) is the number of decimal places up to which X is to be rounded. If D is not provided, X is rounded to the nearest integer and returned.

Database Management Systems

Using Nested Queries



Query: SELECT Dno, avg_salary
FROM (SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno)
AS dept_avg_salary(Dno, avg_salary)
WHERE avg_salary > 32000;

Output:

Dno	avg_salary
1	55000.00
5	33250.00

SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno

- This subquery returns department numbers and corresponding average employee salaries(rounded to 2 decimal places). The output of this subquery is shown below:

Dno	avg_salary
1	55000.00
4	31000.00
5	33250.00

AS dept_avg_salary(Dno, avg_salary)

- This part of the query is used to specify the names of the result relation and its attributes. Hence, the subquery result relation is named *dept_avg_salary*, with the attributes *Dno* and *avg_salary*. Note that failing to the name this result will throw an error:

Error Code: 1248. Every derived table must have its own alias

WHERE avg_salary > 32000

- The WHERE clause selects those tuples from *dept_avg_salary* whose *avg_salary* exceed 32000.

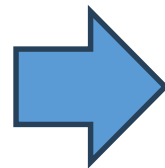
SELECT Dno, avg_salary

- The SELECT clause selects the specified attributes *Dno*, *avg_salary* from the tuples selected by the WHERE clause.

- **Nested subqueries in the FROM clause are supported by most but not all SQL implementations.**
- Some SQL implementations, notably **MySQL** and **PostgreSQL**, require that each subquery relation in the FROM clause must be given a name.
- Oracle allows a subquery result relation to be given a name but does not allow renaming the attributes of the relation.
- Modifications to the written MySQL query can overcome this restriction.

MySQL

```
SELECT Dno, avg_salary  
FROM  
(SELECT Dno, ROUND(AVG (salary),2)  
FROM EMPLOYEE  
GROUP BY Dno)  
AS dept_avg_salary(Dno, avg_salary)  
WHERE avg_salary > 32000;
```



Oracle

```
SELECT Dno, avg_salary  
FROM  
(SELECT Dno, ROUND(AVG (salary),2) AS avg_salary  
FROM EMPLOYEE  
GROUP BY Dno) dept_avg_salary  
WHERE avg_salary > 32000;
```

Database Management Systems

Subqueries in the From clause - Example



Example: Find the maximum of total employee salaries of each department across all departments.

Query: `SELECT MAX(tot_salary)
FROM (SELECT Dno, SUM(Salary) FROM EMPLOYEE GROUP BY Dno)
AS dept_total(Dno, tot_salary);`

Output:

MAX(tot_salary)
133000.00

1) What does the IN comparison operator essentially check for when used with a nested query?

- A) Arithmetic equivalence
- B) Set membership
- C) Partial string matching
- D) Null values

2) If a nested query returns a table with a single attribute and a single tuple, we can treat the result as a _____ and apply operations like the = operator instead of the IN operator.

3) In a nested query with multiple levels, if there is an ambiguity among attribute names between the outer and inner queries, what is the default SQL rule for an unqualified attribute reference?

- A) It refers to the relation declared in the outermost query.
- B) It throws an immediate syntax error.
- C) It refers to the relation declared in the innermost query.
- D) It refers to the first relation listed in the FROM clause.

4) The comparison operator combination = SOME (or = ANY) is logically equivalent to the IN operator, whereas the combination <> ALL is logically equivalent to the _____ operator.

5) When using a subquery expression in the FROM clause in MySQL or PostgreSQL, what strict requirement must be followed to avoid an error (such as Error Code: 1248)?

- A) The subquery must return only a single row.
- B) The subquery must not contain aggregate functions.
- C) Each subquery relation in the FROM clause must be given a name (an alias).
- D) The subquery cannot contain a WHERE clause.

6) The comparison operator $> ALL$ evaluates to True only if the single value is greater than every element in the set, whereas $> SOME$ evaluates to True if the value is greater than _____ element in the set.

Answers:

1. B
2. Scalar
3. C
4. NOT IN
5. C
6. at least one



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Correlated Nested Query

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A **correlated nested query** can be defined as a nested query in which the WHERE clause references **attributes of a relation declared in the outer query**.
- A correlated nested query is **evaluated once for each tuple** (or combination of tuples) of the relation in the outer query.

Example: Retrieve the name of each employee who has a dependent who is of the same gender as the employee.

Query: SELECT E.Fname, E.Lname FROM EMPLOYEE AS E WHERE E.Ssn IN
(SELECT D.Essn FROM DEPENDENT AS D WHERE E.Gender = D.Gender);

Output:

Fname	Lname
Franklin	Wong

- For each *EMPLOYEE* tuple, the nested query is evaluated, and the *Essn* values of all *DEPENDENT* tuples with the same *Gender* as that of the *EMPLOYEE* tuple are retrieved.
- If the *Ssn* value of the *EMPLOYEE* tuple is in the result of the nested query, the *EMPLOYEE* tuple is selected
- In general, a query written with nested SELECT-FROM-WHERE blocks and using the = or IN comparison operators can always be expressed as a single block query.
- For the current example, the equivalent query without a nested query is given below:

```
SELECT DISTINCT E.Fname, E.Lname  
FROM EMPLOYEE AS E, DEPENDENT AS D  
WHERE E.Ssn = D.Essn AND E.Gender = D.Gender;
```

- **EXISTS** and **UNIQUE** are Boolean functions that can be used in the WHERE clause
- The EXISTS function in SQL is used to check whether the **result of a nested query is empty** (contains no tuples) or not.
- **The result of EXISTS is True if the nested query result contains at least one tuple, and False if the nested query result contains no tuples.**
- On the other hand, NOT EXISTS returns True if there are no tuples in the result of the nested query, and False if the nested query result contains at least one tuple.
- EXISTS and NOT EXISTS are typically used in conjunction with a correlated nested query.

Example: Retrieve the name of each employee who has a dependent who is of the same gender as the employee.

Query: SELECT E.Fname, E.Lname FROM EMPLOYEE AS E
WHERE EXISTS (SELECT * FROM DEPENDENT AS D
WHERE E.Ssn = D.Essn AND E.Gender = D.Gender);

Output:

Fname	Lname
Franklin	Wong

- For each *EMPLOYEE* tuple, the nested query is evaluated and it returns all *DEPENDENT* tuples with the same *Essn* and *Gender* as the *EMPLOYEE* tuple.
- If at least one tuple exists in the result of the nested query, then the *EMPLOYEE* tuple under consideration is selected.

Example: Retrieve the names of employees who have no dependents

Query: SELECT Fname, Lname FROM EMPLOYEE

WHERE NOT EXISTS (SELECT * FROM DEPENDENT WHERE Ssn = Essn);

Output:

Fname	Lname
Joyce	English
Ramesh	Narayan
James	Borg
Ahmed	Jabbar
Alicia	Zelaya

- For each *EMPLOYEE* tuple, the correlated nested query selects all *DEPENDENT* tuples whose *Essn* value matches the *Ssn* value in the *EMPLOYEE* tuple
- If the result is empty (i.e. no dependents), the *EMPLOYEE* tuple is selected as the condition in the WHERE clause evaluates to True.

Example: List the names of managers who have at least one dependent.

Query: SELECT Fname, Lname FROM EMPLOYEE
WHERE EXISTS (SELECT * FROM DEPENDENT WHERE Ssn = Essn)
AND EXISTS (SELECT * FROM DEPARTMENT WHERE Ssn = Mgr_ssn);

Output:

Fname	Lname
Franklin	Wong
Jennifer	Wallace

- For each *EMPLOYEE* tuple, the first nested query selects all *DEPENDENT* tuples related to the *EMPLOYEE* tuple.
- The second nested query selects all *DEPARTMENT* tuples managed by *EMPLOYEE* tuple.
- The *EMPLOYEE* tuple is selected if both the nested queries return at least one tuple each.

Example: Retrieve the name of each employee who works on all the projects controlled by department number 4

There are two ways to write this query

Query: SELECT Fname, Lname FROM EMPLOYEE
WHERE NOT EXISTS ((SELECT Pnumber FROM PROJECT WHERE Dnum = 4)
EXCEPT (SELECT Pno FROM WORKS_ON WHERE Ssn = Essn));

Output:

Fname	Lname
Ahmed	Jabbar
Alicia	Zelaya

(SELECT Pnumber FROM PROJECT WHERE Dnum = 4)

- This nested query retrieves the project number of all projects managed by department 4. It is not correlated with the outer query.

(SELECT Pno FROM WORKS_ON WHERE Ssn = Essn)

- This nested query is correlated with the outer query. For each *EMPLOYEE* tuple, it retrieves the project number of all projects that the employee works on.

((SELECT Pnumber FROM PROJECT WHERE Dnum = 4)

EXCEPT (SELECT Pno FROM WORKS_ON WHERE Ssn = Essn));

- Set difference operation is performed between the results of the nested queries i.e. set of project numbers under department 4 MINUS set of project numbers that a particular employee works on.
- The result is the set of project numbers controlled by department 4 that the employee under consideration does not work on.
- If that particular employee works on all projects controlled by department 4, the result of the above query would be an empty relation.
- The condition in the WHERE clause would be satisfied and the *EMPLOYEE* tuple under consideration would be selected

The second way to write this query

Query: SELECT Fname, Lname FROM EMPLOYEE E
WHERE NOT EXISTS (SELECT * FROM WORKS_ON B
WHERE (B.Pno IN (SELECT Pnumber FROM PROJECT
WHERE Dnum = 4)
AND NOT EXISTS (SELECT * FROM WORKS_ON C
WHERE C.Essn = Ssn AND C.Pno = B.Pno)));

Output:

Fname	Lname
Ahmed	Jabbar
Alicia	Zelaya

```
SELECT Fname, Lname FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT * FROM WORKS_ON B
                    WHERE ( B.Pno IN ( SELECT Pnumber FROM PROJECT WHERE Dnum = 4 )
                    AND NOT EXISTS ( SELECT * FROM WORKS_ON C
                                    WHERE C.Essn = Ssn AND C.Pno = B.Pno )));
```

Each color represents a level of nesting

Let us try to understand how this query works:

- There is an **outer query** with 2 levels of nested subqueries. The **outermost nested query** is correlated with the **outer query**. There are **two inner subqueries** nested in the **outermost nested subqueries** – one which is correlated with the **outermost nested query** and the other which is not.

- The **outermost nested query** is evaluated for each *EMPLOYEE* tuple.
- As mentioned, one of the **inner nested queries** is not correlated with the **outermost nested query**:

(SELECT Pnumber FROM PROJECT WHERE Dnum = 4)

- This query returns the project number of all projects controlled by department 4.
- The other inner nested query is correlated and is therefore evaluated for every *WORKS_ON* tuple in the **outermost nested query**:

(SELECT * FROM WORKS_ON C WHERE C.Essn = Ssn AND C.Pno = B.Pno)

- For each *EMPLOYEE* tuple and *WORKS_ON* tuple, this query selects all tuples from the *WORKS_ON* relation whose *Essn* value matches the *Ssn* of the *EMPLOYEE* tuple and project number matches the *Pno* of the outer *WORKS_ON* tuple.

- Now, let's look at the **outermost nested query** (evaluated for every *EMPLOYEE* tuple) :

```
( SELECT * FROM WORKS_ON B WHERE ( B.Pno IN  
( SELECT Pnumber FROM PROJECT WHERE Dnum = 4 )  
AND NOT EXISTS ( SELECT * FROM WORKS_ON C  
WHERE C.Essn = Ssn AND C.Pno = B.Pno )));
```

- This query selects only those *WORKS_ON* tuples that are concerned with projects controlled by department 4 and which the employee under consideration does not work on.
- If the employee under consideration works on all projects under department 4, then there would be no tuples selected by the **outermost nested query**. This would evaluate the WHERE predicate to True (as we are using NOT EXISTS) and hence, the *EMPLOYEE* tuple under consideration would be selected

1) What distinguishes a correlated nested query from a standard (uncorrelated) nested query?

- A) It cannot use the EXISTS operator.
- B) Its WHERE clause references attributes of a relation declared in the outer query.
- C) It executes only once for the entire query statement.
- D) It must always be placed within the FROM clause.

2) Fill in the blank: Unlike a standard subquery, a correlated nested query is evaluated once for each _____ of the relation in the outer query.

3) When using Boolean functions in SQL, what does the EXISTS function evaluate to if its nested query result contains no tuples?

- A) TRUE
- B) UNKNOWN
- C) FALSE
- D) NULL

4) The Boolean functions EXISTS and NOT EXISTS check whether the result of a subquery is empty or not, and they are typically used in conjunction with a _____ nested query.

5) When utilizing the EXCEPT operator to perform a set difference between two nested queries, what happens if the first set is completely subtracted by the second set (e.g., an employee works on all the required projects)?

- A) It returns a NULL value.
- B) It throws a syntax error.
- C) It returns all tuples from the first set.
- D) It results in an empty relation.

Answers:

1. B
2. Tuple
3. C
4. Correlated
5. D



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Views and Users

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A view in MySQL is a virtual table.
- It doesn't store actual data by itself but instead stores a query that pulls data from one or more tables (or even other views)
- Primarily used to hide complex queries and simplify into a convenient table like storage
- Since this is stored in memory, we can reuse these views and avoid writing SQL queries multiple times.
- Generally used when we want a table for specific use case or want to assign roles (Example given in further slides)

- To create a view, we can select fields from one or more tables present in the database.
- A view can either have specific rows based on a certain condition or all the rows of a table.
- We define a view in SQL by using the CREATE VIEW command.

CREATE VIEW view1 AS <query expression>;

where <query expression> is any legal query expression.

The view name is represented by view1.

- We can create a view from one table or using joins with multiple tables as well
- If you create a multi join view then only **read** operations are allowed

NOTE: A view is **updatable** (CRUD operations) only if these criterias are met:

- The view is created on a single table only
- Does not contain aggregate functions like sum, max, count etc
- Does not contain distinct, group by, having or union
- Does not have joins or subqueires (query within query like select for example)
- Query is not recursive in nature

Table Schema

Field	Type	Null	Key	Default	Extra
Author_ID	varchar(50)	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Age	int	YES		NULL	
Email	varchar(50)	YES	UNI	NULL	
Bio	text	YES		NULL	
Password	varchar(50)	NO		NULL	

Assume that there is a table called **Authors** with the following Schema and values. We see how views work using this table alone (crud operations)

Table Values

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	35	alice.johnson@example.com	Expert in AI and ML	password123
A002	Bob Smith	40	bob.smith@example.com	Data Scientist	password456
A003	Carol White	28	carol.white@example.com	Researcher in NLP	password789
A004	David Brown	50	david.brown@example.com	Professor in Quantum Computing	password321
A005	Eve Davis	32	eve.davis@example.com	Cloud Computing Specialist	password654
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A007	Grace Lee	30	grace.lee@example.com	Cryptography Enthusiast	password147
A008	Henry Wilson	27	henry.wilson@example.com	Cybersecurity Analyst	password258
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369

Query

```
create view experienced_authors as  
select * from authors  
where age >= 30 and Bio like '%Expert%';
```

```
select * from experienced_authors;
```

Let's create a view to only check experienced authors who are experts (Age ≥ 30 and have the word Expert in their bio)

Result

```
13 • select * from experienced_authors;  
14
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 						
	Author_ID	Name	Age	Email	Bio	Password
▶	A001	Alice Johnson	35	alice.johnson@example.com	Expert in AI and ML	password123
	A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
	A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369

Query

```
-- Inserting a new experienced author
insert into experienced_authors (Author_ID, Name, Age, Email, Bio, Password)
values ("A010", "John Wick", 52, "john.wick@example.com", "Expert in Pencils", "password567");

-- Updating Alice's Age to 40 from 35
update experienced_authors set Age=40 where Author_ID='A001';
```

Performing insert and update operations on the view (Alice age change and john wick added)

Result

```
22 -- Updated Table with changes
23 • select * from experienced_authors;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 						
Author_ID	Name	Age	Email	Bio	Password	
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123	
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987	
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369	
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567	

Query

```
46  -- Deleting Ivy Scott (A009) from the view
47  • delete from experienced_authors where Author_ID="A009" and Name="Ivy Scott";
48  • select * from experienced_authors;
```

Result Grid Filter Rows: Exports: Wrap Cell Contents:						
Author_ID	Name	Age	Email	Bio	Password	
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123	
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987	
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567	

Deleting Ivy Scott from the view

Authors Table

```
50  -- All the crud operations are reflected on the original table as well
51  • select * from authors;
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Contents:						
Author_ID	Name	Age	Email	Bio	Password	
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123	
A002	Bob Smith	40	bob.smith@example.com	Data Scientist	password456	
A003	Carol White	28	carol.white@example.com	Researcher in NLP	password789	
A004	David Brown	50	david.brown@example.com	Professor in Quantum Computing	password321	
A005	Eve Davis	32	eve.davis@example.com	Cloud Computing Specialist	password654	
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987	
A007	Grace Lee	30	grace.lee@example.com	Cryptography Enthusiast	password147	
A008	Henry Wilson	27	henry.wilson@example.com	Cybersecurity Analyst	password258	
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567	

NOTE: Please note all the changes made to the view is also reflected on the original tables as well
Alice's age has been changed to 40, John wick was added and Ivy Scott was removed from authors table

Example : Create a view that lists each employee's full name, their department name, and the total number of projects they are working on. Then, using the view, write a query to retrieve the names of employees who are working on at least 3 projects.

Query to create a view:

```
CREATE VIEW EmpProjSummary AS
```

```
SELECT E.Fname, E.Lname, D.Dname, COUNT(W.Pno) AS ProjectCount
```

```
FROM EMPLOYEE E JOIN DEPARTMENT D ON E.Dno = D.Dnumber
```

```
JOIN WORKS_ON W ON E.Ssn = W.Essn GROUP BY E.Ssn, E.Fname, E.Lname, D.Dname;
```


View created:

```
mysql> select * from EmpProjSummary;
```

Fname	Lname	Dname	ProjectCount
Jennifer	Wallace	Administration	2
Ahmed	Jabbar	Administration	2
Alicia	Zelaya	Administration	2
James	Borg	Headquarters	1
John	Smith	Research	2
Franklin	Wong	Research	4
Joyce	English	Research	2
Ramesh	Narayan	Research	1

Query the view created:

```
SELECT Fname, Lname, Dname, ProjectCount
```

```
FROM EmpProjSummary
```

```
WHERE ProjectCount >= 3;
```

Fname	Lname	Dname	ProjectCount
Franklin	Wong	Research	4

Example : Create a view that lists employees along with the number of dependents they have. Then, find employees who have more than 2 dependents.

Query to create a view:

```
CREATE VIEW EmpDependentCount AS
```

```
SELECT E.Fname, E.Lname, COUNT(Dp.Dependent_name) AS NumDependents
```

```
FROM EMPLOYEE E
```

```
LEFT JOIN DEPENDENT Dp ON E.Ssn = Dp.Essn
```

```
GROUP BY E.Ssn, E.Fname, E.Lname;
```

View created:

```
mysql> select * from EmpDependentCount;
```

Fname	Lname	NumDependents
John	Smith	3
Franklin	Wong	3
Joyce	English	0
Ramesh	Narayan	0
James	Borg	0
Jennifer	Wallace	1
Ahmed	Jabbar	0
Alicia	Zelaya	0

Query the view created:

```
SELECT * FROM EmpDependentCount
```

```
WHERE NumDependents > 2;
```

Fname	Lname	NumDependents
John	Smith	3
Franklin	Wong	3

- **Complexity:** Views help to reduce complexity. Different views can be created on the same base table for different users.
- **Security:** It increases security by excluding sensitive information from the view.
- **Query Simplicity:** It helps to simplify commands from the user. A view can draw data from several different tables and present it as a single table.
- **Consistency:** A view can present a consistent, unchanged image of the structure of the database. Views can be used to rename columns without affecting the base table.
- **Data Integrity:** If data is accessed and entered through a view, the DBMS can automatically check the data to ensure that it meets the specified integrity constraints.
- **Storage Capacity:** Views take very little space to store the data.
- **Logical Data Independence:** Views can make the application and database tables to a certain extent independent.

- You cannot INSERT if the base table has any not null columns that do not appear in the view.
- You cannot INSERT or UPDATE if any of the columns referenced in the INSERT or UPDATE contains group functions or columns defined by an expression.
- You can't execute INSERT, UPDATE, DELETE statements on a view if with READ ONLY option is enabled.
- You can't create a view using temporary tables.
- You cannot INSERT, UPDATE, or DELETE if the view contains group functions GROUP BY, DISTINCT, or a reference to a pseudo column row_num.
- You can't pass parameters to the SQL server views.
- You can't associate rules and defaults with views.

Database Management Systems

Difference between Views and CTE

Feature	Views	CTEs (Common Table Expressions)
Definition	A virtual table stored in the database, defined by a query.	A temporary result set defined within a query using WITH.
Persistence	Persisted in the database schema until explicitly dropped.	Exists only for the duration of the query execution.
Reusability	Can be reused across multiple queries once created.	Can only be used inside the single query where it is defined.
Scope	Global (available to all users with permissions).	Local (visible only within that query).
Complexity Handling	Good for hiding complex joins, filters, and calculations for repeated use.	Good for breaking down complex queries into readable parts (step-by-step).
Updatability	Some views are updatable (if based on a single table without aggregations).	CTEs are not updatable – they're just temporary results.
Syntax	<pre>CREATE VIEW view_name AS <query>; SELECT * FROM view_name;</pre>	<pre>WITH cte_name AS (<query>) SELECT * FROM cte_name;</pre>

In SQL, "role" and "user" are two distinct concepts used for managing access and permissions within a database management system.

User:

- A user is an individual or entity that interacts with the database.
- Users are typically associated with specific individuals or applications that need to perform operations on the database.
- Each user has a unique username and, in some database systems, a password for authentication. Users are granted permissions to perform specific actions (e.g., SELECT, INSERT, UPDATE, DELETE) on database objects (e.g., tables, views).

Role:

- A role is a named group of users or other roles.
- Roles are used to simplify and manage permissions and access control in a database. Instead of granting permissions to individual users, you can assign permissions to roles.
- Users are then added to roles, and they inherit the permissions associated with those roles. This makes it easier to manage and change permissions for multiple users at once.
- Roles can also be nested, meaning you can have roles within roles, creating a hierarchical permission structure.

Here's how roles and users are typically used together:

- You create roles and assign specific permissions to these roles. For example, you might create a "Readers" role with only SELECT permissions and a "Writers" role with INSERT, UPDATE, and DELETE permissions on certain tables.
- You add individual users to the roles they need. For instance, you might add UserA to the "Readers" role and UserB to the "Writers" role.
- Users then inherit the permissions associated with the roles they belong to. So, UserA will have SELECT permissions because they are in the "Readers" role, and UserB will have INSERT, UPDATE, and DELETE permissions because they are in the "Writers" role.

Syntax for Create User:

`CREATE USER 'username'@'host' IDENTIFIED BY 'password';`

- 'username' : the account name.
- 'host' : where the user can connect from:
 - 'localhost' : only local machine.
 - '%' : any host.
 - Specific IP or hostname possible.
- 'password' : the login password for authentication.

Example: **`CREATE USER 'student'@'localhost' IDENTIFIED BY 'stud123';`**

To delete the created user:

`DROP USER 'username'@'host';`

Example: `drop user 'student'@'localhost';;`

- MySQL has a feature that provides many control options to the administrators and users on the database.
- We have already learned how to create a new user using the CREATE USER statement in the MySQL.
- Now, we are going to learn about granting privileges to a user account. MySQL provides GRANT statements to give access rights to a user account.
- The SQL standard includes the privileges SELECT, INSERT, UPDATE, and DELETE.

Syntax: **GRANT privileges**
ON database.table
TO 'username'@'host';

Privileges : what the user can do (SELECT, INSERT, UPDATE, etc.).

Database.table : the scope:

- ***.*** : all databases, all tables.
- **mydb.*** : all tables in mydb.
- **mydb.mytable** : specific table.

username@host : user account.

Example: grant all on company to student@localhost;

//student user should be created, the above command grants all privileges to student user

Common Privileges

- **Data privileges:**
 - SELECT, INSERT, UPDATE, DELETE
- **Structure privileges:**
 - CREATE, ALTER, DROP, INDEX
- **Administrative privileges:**
 - GRANT OPTION allows the user to grant privileges to others
 - SUPER is for superuser permissions
 - ALL PRIVILEGES is a shorthand for all available privileges.

Examples: school is the database and students is a table

a) Give read-only access on one table to the user

```
GRANT SELECT ON school.students TO 'alice'@'localhost';
```

a) Give full access on a database to the user

```
GRANT ALL PRIVILEGES ON school.* TO 'bob'@'localhost';
```

a) To check the privileges assigned to a specific user, use the following command

```
SHOW GRANTS FOR 'bob'@'localhost';
```

To revoke an authorization, we use the REVOKE statement.

It takes a form almost identical to that of GRANT.

Syntax:

```
REVOKE privileges  
ON database.table  
FROM 'username'@'localhost';
```

Example:

```
REVOKE select
```

```
ON school.*
```

```
FROM 'alice'@'localhost';
```


Any authorization that can be granted to a user can be granted to a role.

Roles are granted to users just as authorizations are.

Roles can be created in SQL as follows:

```
create role Assistant_employee;
```

Roles can then be granted privileges just as the users :

```
grant select
```

```
on courses
```

```
to Assistant_employee;
```

Roles can be granted to users, as well as to other roles:

```
grant rolename to user_name;
```

Example: grant Assistant_employee to 'alice'@'localhost';

Revoking privileges from roles:

```
REVOKE INSERT, UPDATE, DELETE ON database_name FROM role_name;
```

Removing roles:

To delete one or more roles, you use the DROP ROLE statement as follows:

```
DROP ROLE role_name[, role_name, ...];
```

```
DROP ROLE role1, role2;
```

1) Which of the following best describes a "view" in a database?

A) A physical table that stores a duplicate copy of data to speed up queries.

B) A temporary result set defined within a single query using the WITH clause.

C) A virtual table that does not store actual data by itself but instead stores a query that pulls data from one or more tables.

D) A specialized role assigned to users for administrative privileges.

2) Fill in the blank: While a View is persisted in the database schema until it is explicitly dropped, a _____ exists only for the duration of a single query execution.

3) According to the slides, under which condition is a view considered "updatable" (meaning it allows CRUD operations like INSERT or UPDATE)?

A) When it contains aggregate functions like SUM or MAX.

B) When it is created using joins across multiple tables.

C) When it is created on a single table only and does not contain subqueries or the DISTINCT keyword.

D) When it includes a GROUP BY or HAVING clause.

4) Fill in the blank: While the GRANT statement is used to give access rights to a user account, the _____ statement is used to remove an authorization previously given to a user or role.

5) How do views provide increased security in a database environment?

- A) By automatically encrypting all passwords stored in the base tables.
- B) By requiring a secondary username and password every time the view is accessed.
- C) By excluding sensitive information from the view, thereby restricting what data the user can see.
- D) By permanently locking the base tables from any INSERT or UPDATE commands.

Answers:

- 1. C
- 2. CTE (Common table Expression)
- 3. C
- 4. REVOKE
- 5. C

- <https://www.devart.com/dbforge/mysql/studio/create-view-in-mysql.html>
- <https://thanusan25thanu.medium.com/mysql-views-are-not-only-query-able-but-also-updatable-91ac4730bc11>
- <https://dev.mysql.com/doc/refman/8.4/en/views.html>
- <https://www.red-gate.com/simple-talk/databases/mysql/working-with-mysql-views/>
- <https://sqlpad.io/tutorial/ctes-vs-views-in-sql-essential-distinctions-for-tech-interviews>
- <https://learnsql.com/blog/difference-between-sql-cte-and-view>
- <https://minhngocda.medium.com/view-vs-cte-in-sql-difference-and-use-cases-227307d0d672>



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Triggers

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A trigger is a statement that the system executes automatically as a side effect of a modification to the database.

Defining a trigger involves:

- Specifying when a trigger needs to be executed. This consists of two components:
 - Event** – which causes the trigger to be checked
 - Condition** – which must be satisfied for trigger execution to proceed
- Specifying the **action** to be taken when the trigger executes
- Once we enter a trigger into the database, the system executes it whenever the specified event occurs and the corresponding condition is satisfied.

- **Triggers can be used to implement certain integrity constraints that cannot be specified using the constraint mechanism of SQL.**
- Triggers are also useful mechanisms for alerting humans or for starting certain tasks automatically when certain conditions are met.
- Consider a scenario wherein a student has taken up a new course and a new tuple needs to be inserted in the *TAKES* relation. We could design a trigger that updates the tuple in the *STUDENT* relation for the student taking the course by adding the number of credits for the course to the student's total credits, whenever a new tuple is inserted in the *TAKES* relation.
- Triggers usually cannot perform updates outside the database. However, some database systems provide built-in support for sending email from triggers.
- Triggers can be used in various applications, such as maintaining database consistency, monitoring database updates, and updating derived data automatically.

Triggers - Syntax

Let's have a look at the syntax of the CREATE TRIGGER statement.

Syntax:

```
CREATE TRIGGER trigger_name  
(AFTER | BEFORE) (INSERT | UPDATE | DELETE) ON table_name  
FOR EACH ROW  
BEGIN  
    --variable declarations  
    --trigger code  
END;
```

- The CREATE TRIGGER statement is used to create a trigger with the name *trigger_name* on the relation *table_name*.

Let's try to understand the syntax of the CREATE TRIGGER statement:

CREATE TRIGGER trigger_name

- The first line of the syntax (shown above) is used to **specify the name of the trigger**.

(AFTER | BEFORE) (INSERT | UPDATE | DELETE) ON table_name

- The second line of the syntax is used to specify the **event which causes the trigger to be checked**.
 - table_name - specifies the **relation** which the trigger must check for modification
 - INSERT | UPDATE | DELETE - specifies the **type of modification to be checked** i.e. whether the trigger must be checked on insertion, updation or deletion of tuples.
 - AFTER | BEFORE - specifies **when the trigger must be checked** i.e. before or after the specified relation undergoes modification.

FOR EACH ROW

- Multiple tuples could be modified by a single SQL statement(for eg, multiple tuples could be inserted using one INSERT statement). The third line specifies that the trigger code would explicitly iterate over each modified row.
- The part of the query between the **BEGIN** and **END** keywords corresponds to the **action** to be taken on the modified tuples. This involves declaring any variables that would be used and specifying the SQL statements to be executed. Actions could be the execution of stored procedures as well.

Consider a sample database that stores information about students and the marks obtained by them in various courses. It has three tables:

- *Student_sample*(SRN, name) – stores information about students which includes their name and SRN (a unique identifier for each student).
- *Marks_sample*(SRN, Course, Marks) - stores information about marks obtained by students(identified by their SRN) in different courses.
- *Marks_history*(SRN, Course, Marks) – a backup table that stores information that has been deleted from the *Marks_sample* table.

The entries in the tables are shown below:

Student_sample:

SRN	Name
1	Harry
2	Ron
3	Hermione
4	Draco

Marks_sample:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Note that the Marks_history table is initially empty.

Example: Create a trigger that checks if the entered marks are valid or not whenever an entry is made in the Marks_sample table.

Code:

```
DELIMITER //

CREATE TRIGGER CheckMarks
BEFORE INSERT ON Marks_sample
FOR EACH ROW
BEGIN
    IF NEW.marks < 0 OR NEW.marks > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid marks: Marks must be between 0 and 100';
    END IF;
END;//

DELIMITER ;
```

- The given SQL statement creates a trigger *CheckMarks*. It gets executed **whenever there is an insertion** in the *Marks_sample* relation. The keyword BEFORE specifies that the **trigger execution occurs before the triggering operation** (in this case, insertion of tuples in the *Marks_sample* relation) is executed.
- The trigger action gets executed **for every row that is inserted**. This is specified by the FOR EACH ROW clause.
- The trigger checks if the *Marks* value of the inserted tuple is **lesser than 0 or greater than 100** (i.e. it checks if the **Marks value is invalid**). If the specified condition is satisfied, the trigger raises an **error**, an appropriate **error message** is displayed and the **tuple is not inserted** (trigger execution occurs before insertion). If the condition is not satisfied, the tuple is inserted.

Let's try inserting into the *Marks_sample* relation using the statement:

```
INSERT INTO Marks_sample VALUES (1,'C03',101);
```

BEFORE:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Output:

✖ 13 12:19:02 INSERT INTO Marks_sample VALUES (1,'C03',101)

Error Code: 1644. Invalid marks: Marks must be between 0 and 100

0.000 sec

The tuple does not get inserted as the *Marks* value is greater than 100.

Note that if the *Marks* value is NULL, then the tuple gets inserted.

```
INSERT INTO Marks_sample VALUES (1,'C03',NULL);
```

BEFORE:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

SRN	Course	Marks
1	C01	98
1	C02	89
1	C03	NULL
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Example: Implement a trigger that mimics the behaviour of ON UPDATE CASCADE.

Code:

```
DELIMITER //
CREATE TRIGGER update_marks_on_cascade
BEFORE UPDATE ON Student_sample
FOR EACH ROW
BEGIN
    SET FOREIGN_KEY_CHECKS = 0;
    UPDATE Marks_sample SET SRN = NEW.SRN WHERE SRN = OLD.SRN;
    SET FOREIGN_KEY_CHECKS = 1;
END;
DELIMITER ;
```

NOTE : SET FOREIGN_KEY_CHECKS = 0 disables all foreign key checks(i.e. even if a foreign key constraint violation occurs, an error will not be raised). This is done temporarily to update the *Marks_sample* relation, after which the foreign key checks is enabled by specifying the SET FOREIGN_KEY_CHECKS = 1

- The trigger *update_marks_on_cascade* updates the *SRN* values of tuples in the *Marks_sample* relation which are related to the tuples(in the *Student_sample* relation) being modified.
- To see its effect, let's execute the given query:

UPDATE Student_sample SET SRN = 5 WHERE name = 'Harry';

BEFORE:

Student_sample

SRN	Name
1	Harry
2	Ron
3	Hermione
4	Draco

Marks_sample

SRN	Course	Marks
1	C01	98
1	C02	89
1	C03	NULL
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

Student_sample

SRN	Name
2	Ron
3	Hermione
4	Draco
5	Harry

Marks_sample

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

- The trigger automatically **updates the SRN values of all tuples** in the *Marks_sample* relation that are **related** to the tuple with *name* 'Harry'.

If this trigger would not have been created (and if ON UPDATE CASCADE was not specified in the foreign key constraint in the *Marks_sample* table), then:

- updating the SRN of any tuple in the *Student_sample* relation that have **related tuples** in the *Marks_sample* relation would have **thrown an error** as this updation will violate the foreign key constraint.
- However, updating the SRN of tuples that do not have any related tuples in the *Marks_sample* relation would not throw an error.

Example: Implement a trigger that stores all tuples deleted from the *Marks_sample* table in the *Marks_history* table

Code:

```
DELIMITER //
CREATE TRIGGER backup_marks_info
BEFORE DELETE
ON Marks_sample
FOR EACH ROW
BEGIN
INSERT INTO Marks_history SELECT * FROM Marks_sample WHERE SRN = OLD.SRN AND COURSE =OLD.COURSE;
END //
DELIMITER ;
```


Let's see the effect of this trigger by executing the given query:

```
DELETE FROM Marks_sample WHERE SRN = 2;
```

BEFORE:

Marks_sample

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

Marks_history

SRN	Course	Marks
-----	--------	-------

(empty set)

AFTER:

Marks_sample

SRN	Course	Marks
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

Marks_history

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80

- 1) Which of the following best describes the three primary components involved in defining a database trigger?
 - A) Select, From, Where
 - B) Event, Condition, Action
 - C) Insert, Update, Delete
 - D) Begin, Execute, End

- 2) When defining a trigger, the _____ clause is used to specify that the trigger code should explicitly iterate over every single modified tuple (e.g., when a single INSERT statement modifies multiple rows).

3) Which keyword is used in a CREATE TRIGGER statement to specify that the trigger execution must occur prior to the triggering database operation (such as an insertion or deletion)?

- A) PRECEDING
- B) INSTEAD OF
- C) PRIOR
- D) BEFORE

4) One of the primary needs for triggers is to implement complex _____ constraints that cannot be specified using the standard constraint mechanisms provided by SQL.

5) In the slide demonstrating how to mimic an ON UPDATE CASCADE behavior, which specific command is used inside the trigger to temporarily disable foreign key constraint violations during the update?

- A) SET FOREIGN_KEY_CHECKS = 0;
- B) DISABLE CONSTRAINTS ALL;
- C) ALTER TABLE DISABLE KEYS;
- D) SET CONSTRAINTS = OFF;

Answers:

1. B
2. FOR EACH ROW
3. D
4. Integrity
5. A



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Functions

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Functions



- We have already seen several functions that are built into the SQL language.
- In this section, we show how developers can write their own functions and procedures, store them in the database, and then invoke them from SQL statements.
- Functions and procedures allow “business logic” to be stored in the database and executed from SQL statements.
- These can be defined either by the procedural component of SQL or by an external programming language such as Java, C, or C++.
- Functions that are created by the user in the system database or a user-defined database are known as user-defined functions.
- The UDF functions accept parameters, perform actions, and returns the result.

Database Management Systems

Functions

- These functions help us to simplify our development by encapsulating complex business logic and making it available for reuse anywhere based on the needs.
- The user-defined functions make the code needed to query data a lot easier to write.
- They also improve query readability and functionality, as well as allow other users to replicate the same procedure.



- function in SQL Server always accepts parameters, either single or multiple and returns a single value or table.
- The functions are useful in the simplification of our code.
- Suppose we might have a complex computation that appears in a number of queries.
- In such a case, we can build a function that encapsulates the formula and uses it in each query instead of in each query.

Syntax:

```
CREATE FUNCTION schema_name.function_name (parameter_list)
RETURNS data_type AS
BEGIN
    statements
    RETURN value
END
```

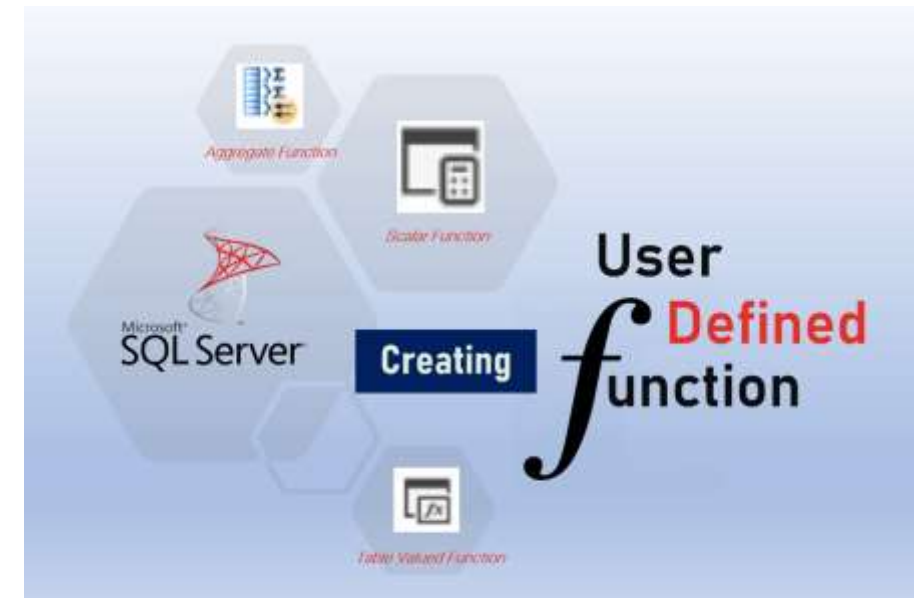
Database Management Systems

Functions

- **function_name** : It is the name of the function that needs to be created in MySQL.
- **parameter1, parameter2,...** : We can pass the optional parameters to the functions that need to be declared while creating it in the () brackets.
- **A function can contain none, one or more than one parameter.**
- **BEGIN and END** – BEGIN keyword marks the beginning of the function while END marks the completion of function in MYSQL.

How to Delete function?

```
DROP FUNCTION IF EXISTS function_name;
```



- **RETURN Datatype** – We can return any type of value from the execution of the function.
- The type of value that will be returned needs to be specified after the RETURN clause.
- Once, MySQL finds the RETURN statement while execution of the function, execution of the function is terminated and the value is returned.
- **DETERMINISTIC** – The function can be either deterministic or non-deterministic which needs to be specified here.
- When the function returns the same output for the same input of parameter then it is called deterministic. However, if the function returns a different output for the same input of functions then we can call that function to be nondeterministic.
- Functions will be NON-DETERMINISTIC by default.

Database Management Systems

Functions



Example:

```
CREATE FUNCTION Dept_size(deptno INT)
  RETURNS VARCHAR(7)
  BEGIN
    DECLARE No_of_ems INT;
    SELECT COUNT(*) INTO No_of_ems FROM EMPLOYEE WHERE Dno = deptno;
    IF No_of_ems > 3 THEN RETURN 'HUGE';
    ELSEIF No_of_ems > 2 THEN RETURN 'LARGE';
    ELSEIF No_of_ems > 1 THEN RETURN 'MEDIUM';
    ELSE RETURN 'SMALL';
    END IF;
  END
```

Query: select Dname,Dnumber,Dept_size(Dnumber)from department;

Output:

Dname	Dnumber	Dept_size(Dnumber)
Administration	4	HUGE
Headquarters	1	SMALL
Research	5	HUGE

Database Management Systems

Functions



Example: Write a function to find the age of employees

```
DELIMITER //

CREATE FUNCTION no_of_years(date1 date)
RETURNS INTEGER deterministic

BEGIN

    DECLARE date2 DATE;

    Select current_date()into date2;

    RETURN year(date2)-year(date1);

END //

DELIMITER;
```

Output:

Fname	no_of_years(Bdate)
John	58
Franklin	68
Joyce	51
Richard	61
Ramesh	61
James	86
Jennifer	82
Ahmed	54
Alicia	55

Query: select Fname,no_of_years(Bdate) from employee;

- 1) What is the default classification of user-defined functions in MySQL when neither DETERMINISTIC nor NON-DETERMINISTIC is explicitly specified?
 - A) DETERMINISTIC
 - B) NON-DETERMINISTIC
 - C) STATIC
 - D) SCALAR

- 2) A user-defined function that always returns the same output for the exact same input parameters is referred to as a _____ function.

3) Which statement is used to safely remove an existing user-defined function from the database if it exists?

- A) DELETE FUNCTION function_name;
- B) REMOVE FUNCTION function_name;
- C) DROP FUNCTION IF EXISTS function_name;
- D) CLEAR FUNCTION function_name;

4) Once MySQL encounters a _____ statement during the execution of a function, the execution of the function is immediately terminated and the value is returned to the caller.

Answers:

1. B
2. deterministic
3. C
4. RETURN



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems Procedures

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A procedure (often called a stored procedure) is a collection of pre-compiled SQL statements stored inside the database.
- It is a subroutine or a subprogram in the regular computing language.
- A procedure always contains a name, parameter lists, and SQL statements. We can invoke the procedures by using triggers, other procedures and applications such as Java , Python , PHP , etc.
- The following syntax is used for creating a stored procedure in MySQL.

```
DELIMITER &&
CREATE PROCEDURE procedure_name [[IN | OUT | INOUT] parameter_name datatype
    [, parameter      datatype]) ]
    BEGIN
        Declaration_section
        Executable_section
    END &&
DELIMITER ;
```

Stored procedures are useful in the following circumstances:

- If a database program is needed by several applications, it can be stored at the server and invoked by any of the application programs.
- This reduces duplication of effort and improves software modularity.
- Executing a program at the server can reduce data transfer and communication cost between the client and server in certain situations.
- These procedures can enhance the modeling power provided by views by allowing more complex types of derived data to be made available to the database users via the stored procedures.
- Additionally, they can be used to check for complex constraints that are beyond the specification power of assertions and triggers.

Modes of Procedure Parameters:

- **IN:** It is the default mode. It takes a parameter as input, such as an attribute. When we define it, the calling program has to pass an argument to the stored procedure. This parameter's value is always protected.
- **OUT:** It is used to pass a parameter as output. Its value can be changed inside the stored procedure, and the changed (new) value is passed back to the calling program. It is noted that a procedure cannot access the OUT parameter's initial value when it starts.
- **INOUT:** It is a combination of IN and OUT parameters. It means the calling program can pass the argument, and the procedure can modify the INOUT parameter, and then passes the new value back to the calling program.
- How to call a stored procedure?
- Syntax: `CALL procedure_name ( parameter(s))`

Consider this table

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975
4	104	Barack	maths	90	8769875325
5	105	Rinky	maths	85	6753157975
6	106	Adam	Science	92	7964225686
7	107	Andrew	Science	83	5674243757
8	108	Brayan	Science	85	7523416567
10	110	Alexandar	Biology	67	2347346438

Procedure without Parameter:

Example: Suppose we want to display all records of this table whose marks are greater than 70 and count all the table rows.

The following code creates a procedure named get_merit_students:

```
DELIMITER $$
```

```
CREATE PROCEDURE`get_merit_student`()
```

```
BEGIN
```

```
    SELECT * FROM student WHERE marks > 70;
```

```
    SELECT COUNT(student_code) AS Total_Student FROM student;
```

```
END$$
```

```
DELIMITER ;
```

Query: CALL get_merit_student();

Output:

Total_Student
9

Procedures with IN Parameter

In this procedure, we have used the IN parameter as '**var1**' of integer type to accept a number from users. Its body part fetches the records from the table using a **SELECT** statement and returns only those rows that will be supplied by the user. It also returns the total number of rows of the specified table. See the procedure code:

```
DELIMITER &&
```

```
CREATE PROCEDURE get_student (IN var1 INT)
```

```
BEGIN
```

```
    SELECT * FROM student_info LIMIT var1;
```

```
    SELECT COUNT(student_code) AS Total_Student FROM student_info;
```

```
END &&
```

```
DELIMITER ;
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975

Query:

Call get_student(3);

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975

Query:

Call get_student(5);

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975
4	104	Barack	maths	90	8769875325
5	105	Rinky	maths	85	6753157975

Procedures with OUT Parameter

In this procedure, we have used the OUT parameter as the '**highestmark**' of integer type. Its body part fetches the maximum marks from the table using a **MAX() function**. See the procedure code:

```
DELIMITER &&
```

```
CREATE PROCEDURE `display_max_mark` (OUT highestmark INTEGER)
```

```
BEGIN
```

```
    SELECT max(marks) INTO highestmark FROM student;
```

```
END&&
```

```
DELIMITER ;
```

Query:

```
call display_max_mark(@output);
```

```
select @output;
```

Output:

@output
92

Procedures with INOUT Parameter:

In this procedure, we have used the INOUT parameter as '**var1**' of integer type. Its body part first fetches the marks from the table with the specified **id** and then stores it into the same variable var1. The var1 first acts as the IN parameter and then OUT parameter. Therefore, we can call it the INOUT parameter mode. See the procedure code:

```
DELIMITER &&
```

```
CREATE PROCEDURE `display_marks` (INOUT var1 INTEGER)
```

```
BEGIN
```

```
    SELECT marks INTO var1 FROM student WHERE stud_id = var1;
```

```
END&&
```

```
DELIMITER ;
```

Query:

```
SET @M = '3';  
CALL display_marks(@M);  
SELECT @M;
```

Output:

@M
70

Query:

```
SET @M = '4';  
CALL display_marks(@M);  
SELECT @M;
```

Output:

@M
90

How to delete/drop stored procedures in MySQL?

MySQL also allows a command to drop the procedure. When the procedure is dropped, it is removed from the database server also. The following statement is used to drop a stored procedure in MySQL:

DROP PROCEDURE [IF EXISTS] procedure_name;

Ex: DROP PROCEDURE display_marks;

We can verify it by listing the procedure in the specified database using the SHOW PROCEDURE STATUS command.

SHOW PROCEDURE STATUS [LIKE 'pattern' | WHERE search_condition]

Ex: SHOW PROCEDURE STATUS WHERE db = 'mystudentdb';


```
CREATE PROCEDURE StudentMarksWithLoop()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE marks INT;
    DECLARE student_name VARCHAR(25);

    WHILE done = 0 DO
        SELECT marks, student_name INTO marks,
            student_name
        FROM student
        WHERE done = 0
        LIMIT 1;
```

```
        IF marks >= 50 THEN
            SELECT CONCAT(student_name, ' passed')
        AS Result;
        ELSE
            SELECT CONCAT(student_name, ' failed')
        AS Result;
        END IF;

        SET done = 1;
    END WHILE;
END //

DELIMITER ;
```

Note: This WHILE loop that continues to process students until the done flag is set to 1. Inside the loop, we retrieve the marks and student_name for one student at a time, check if they passed or failed, and then set the done flag to 1 to exit the loop after processing one student.

STORED PROCEDURE Vs FUNCTION

Stored Procedure	Function
Supports in, out and in-out parameters,i.e., input and output parameters	Supports only input parameters, no output parameters.
Stored procedures can call functions as needed	The function cannot call a stored procedure
There is no provision to call procedures from select/having and where statements	You can call functions from a select statement
Transactions can be used in stored procedures	No transactions are allowed
Can do exception handling by inserting try/catch blocks	No provision for explicit exception handling
Need not return any value	Must return a result or value to the caller
All the database operations like insert, update, delete can be performed	Only select is allowed

1) Which of the following parameter modes is used to pass a parameter as output, where the procedure can change its value and pass it back to the calling program, but cannot access its initial value when the procedure starts?

- A) IN
- B) OUT
- C) INOUT
- D) RETURN

2) To execute or invoke a stored procedure in MySQL from an application or terminal, you must use the _____ keyword followed by the procedure name and its parameters.

3) According to the comparison between Stored Procedures and Functions, which of the following is a capability unique to Stored Procedures?

- A) They must return a single result or value to the caller.
- B) They can be called directly from within SELECT or WHERE statements.
- C) They support the use of transactions and can perform database operations like INSERT, UPDATE, and DELETE.
- D) They only support input parameters and do not allow output parameters.

4) Once a procedure is created, you can verify it by listing the procedures in the specified database using the SHOW PROCEDURE _____ command.

5) What is one of the primary performance benefits of executing a stored procedure directly at the server side instead of processing the logic within the client application?

- A) It completely eliminates the need for database indexing.
- B) It reduces data transfer and communication cost between the client and server.
- C) It allows the database to automatically bypass all authorization checks.
- D) It converts the database into a distributed cluster.

6) While a stored procedure can call functions as needed, there is no provision to call a procedure from a SELECT or WHERE statement; instead, only a _____ can be called directly from a SELECT statement.

Answers:

1. B
2. CALL
3. C
4. STATUS
5. B
6. Function



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Common Table Expressions(CTE)

Anchor:

Dr Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A **Common Table Expression (CTE)** in SQL is a temporary result set that is defined and used within the execution scope of a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement.
- CTEs are defined using the **WITH** clause and can be referenced multiple times within the main SQL query.

They can be useful for the following:

- Simplifying complex queries
- Breaking down subqueries
- Recursive querying

Basic Syntax of a CTE (with clause–recap)

```
WITH cte_name AS ( SELECT query1.....)  
SELECT query2...FROM cte_name;
```

- **cte_name** is the name of the CTE
- **query1** is a valid SQL query that returns a result set, which will be treated as a virtual table within the main query.
- **query2** is an outer query which uses CTE like a table

Can use **multiple CTEs** separated by commas

```
WITH cte1 AS (...), cte2 AS (...)  
SELECT ...  
FROM cte1 JOIN cte2;
```

Simple example of CTE considering Company Database

```
WITH cte_emp AS (  
    SELECT ssn, fname, dno, salary  
    FROM Employee  
    WHERE salary > 50000  -- filter applied here  
)  
SELECT fname FROM cte_emp; --use of cte
```

Retrieve name, dno, salary of all Employees earning above average salary

```
WITH avg_salary AS (  
    SELECT AVG(salary) AS avg_sal  
    FROM Employee  
)  
SELECT fname, dno, salary  
FROM Employee, avg_salary  
WHERE Employee.salary > avg_salary.avg_sal;
```

Query without using “with”

```
select fname,dno,salary from employee where salary>(select  
avg(salary) from employee);
```

Get employees with salary > 40,000 along with their department name (without CTE)

```
select fname,salary,dname from employee join department on dno=dnumber where salary>40000;
```

Get employees with salary > 40,000 along with their department name using CTE

```
WITH cte_emp AS (  
    SELECT ssn, fname, dno, salary FROM Employee  
    WHERE salary > 40000  
)  
cte_dept AS (  
    SELECT dnumber, dname  
    FROM department  
)  
SELECT e.fname, e.salary, d.dname  
FROM cte_emp e JOIN cte_dept d ON e.dno = d.dnumber;
```

- The WITH clause provides a way of defining a temporary relation whose definition is available only to the query in which the WITH clause occurs.

Example: Find the ssn of the employee who works for the highest number of hours on a particular project. Display the project number and the number of hours the employee has worked as well

Query: WITH max_work(max_hours) AS
(SELECT MAX(Hours) FROM WORKS_ON)
SELECT Essn, Pno, Hours
FROM WORKS_ON, max_work
WHERE Hours = max_hours;

Output:

Essn	Pno	Hours
666884444	3	40.0

- The WITH clause in the query defines the temporary relation *max_work*, which is a single tuple-single attribute relation holding the maximum hours that an employee works. The contents of *max_work* are as shown :

max_hours
40.0

- The relation *max_work* is used in the main query. A join is performed between the *WORKS_ON* and *max_work* relations on the condition that the *Hours* value matches *max_hours*. All tuples in the *WORKS_ON* relation with the maximum value of *Hours* get selected(here there is only one such tuple).
- The *Essn*, *Pno*, and *Hours* values of the selected tuples are retrieved.

- Note that the same result could be achieved using nested queries.

Query: SELECT Essn, Pno, Hours

FROM WORKS_ON

WHERE Hours = (SELECT MAX(Hours) FROM WORKS_ON);

Output:

Essn	Pno	Hours
666884444	3	40.0

- However, using nested subqueries would have made the query harder to read and understand.
- The WITH clause makes the query logic clearer. It also permits this temporary relation to be used in multiple places within a query.

Example: Find all departments where the total salary is greater than the average of the total salary at all departments.

Query: WITH dept_total(Dno,tot_salary)

AS(SELECT Dno, SUM(Salary) FROM EMPLOYEE GROUP BY Dno),

dept_total_avg(avg_tot_salary) AS (SELECT AVG(tot_salary) FROM dept_total)

SELECT Dname, Dnumber FROM dept_total, dept_total_avg, DEPARTMENT

WHERE tot_salary > avg_tot_salary AND Dnumber = Dno;

Output:

Dname	Dnumber
Research	5

```
dept_total(Dno,tot_salary) AS(SELECT Dno, sum(Salary)
FROM EMPLOYEE GROUP BY Dno)
```

- This part of the query retrieves department number and total salary for each department and the result is named *dept_total*. It would look like this:

Dno	tot_salary
1	55000.00
4	93000.00
5	133000.00

```
dept_total_avg(avg_tot_salary) AS (SELECT AVG(tot_salary)FROM dept_total)
```

- This part of the query retrieves the average of total salaries in all departments by making use of *dept_total* which was defined previously. The result is named *dept_total_avg*.

- *dept_total* and *dept_total_avg* are then joined on the condition that *tot_salary* is greater than *avg_tot_salary*.
- The result of the previous join is joined with *DEPARTMENT* to obtain the department name, which is not available in the result of the join between *dept_total* and *dept_total_avg*.

Recursive CTEs

- A recursive **CTE** is one that references itself within that **CTE**.
- The recursive **CTE** is useful when working with hierarchical data as the CTE continues to execute until the query returns the entire hierarchy.
- A typical example of hierarchical data is a table that includes a list of **employees** like in the company database employee has recursive relationship, which represents the employee–supervisor relationship.
- In SQL, recursive queries are typically implemented using Common Table Expressions (CTEs) with the WITH RECURSIVE clause.

Here's the general syntax for creating a recursive query

WITH RECURSIVE cte_name (column_list) AS (

-- Anchor member: The initial query that forms the base case

SELECT ...

FROM ...

WHERE ...

UNION ALL

-- Recursive member: The query that refers to the CTE itself

SELECT ...

FROM cte_name **-- Refer to the CTE within itself**

WHERE ...

)-- The final SELECT statement outside the CTE to retrieve the result

SELECT ...FROM cte_name;

Here's an explanation of each part of the syntax:

- **cte_name:** This is the name you give to the Common Table Expression (CTE), which you can use to refer to it within the CTE itself and in the final SELECT statement.
- **column_list:** Specify the list of columns you want to include in the CTE's result set.
- **Anchor member:** This is the initial query that forms the base case of the recursion. It selects the starting point or points for the recursive process. It's important to ensure that this query doesn't reference the CTE itself.

Here's an explanation of each part of the syntax:

- **UNION ALL:** This is used to combine the results of the anchor member with the results of the recursive member. UNION ALL is used because it includes duplicate rows, which is often necessary in recursive queries.
- **Recursive member:** This is the query that refers to the CTE itself, creating the recursion. It selects records based on relationships established in previous iterations. This query must reference the CTE by its name and should include a termination condition to stop the recursion.
- **Final SELECT statement:** This query retrieves the final result set from the CTE. It typically follows the recursive part of the CTE definition.

Example: Given an example of a recursive query that counts down from 5 to 1:

```
WITH RECURSIVE Countdown (n) AS (
```

```
-- Anchor member
```

```
SELECT 5
```

```
UNION ALL
```

```
-- Recursive member
```

```
SELECT n - 1
```

```
FROM Countdown
```

```
WHERE n > 1
```

```
)
```

```
-- Final SELECT statement
```

```
SELECT n FROM Countdown;
```

n
5
4
3
2
1

Example: Given an example of a recursive query that counts down from 1 to 5:

```
WITH RECURSIVE cte(n) AS (  
    -- Anchor part of the query  
    SELECT 1  
    UNION ALL  
    -- Recursive part of the query  
    SELECT n + 1 FROM cte WHERE n < 5  
)  
SELECT * FROM cte;
```

n
1
2
3
4
5

Execution Flow:

A **recursive CTE** has two parts:

1. **Anchor member** → the initial result set
2. **Recursive member** → repeatedly applied until it stops producing rows

The database engine runs it in an iterative fashion.

Run the anchor query first: `SELECT 1`

This produces the first row: 1.

Then, Apply the recursive part - SQL substitutes the current contents of cte into:

```
SELECT n + 1 FROM cte WHERE n < 5
```

From (1), since $1 < 5$, it generates: 2 which is added to the result.

Give a query to extract the organizational hierarchy beginning with a specific employee, typically the CEO, and extending downward through the reporting structure.

```
with recursive orghierarchy as(  
  select ssn,fname,lname,super_ssn, 1 as level  
  from employee  
  where super_ssn is null  
  union all  
  select e.ssn,e.fname,e.lname,e.super_ssn, OH.level+1  
  from employee e  
  join orghierarchy OH on e.super_ssn=OH.ssn  
)  
select * from orghierarchy;
```

Output:

ssn	fname	lname	super_ssn	level
888665555	James	Borg	NULL	1
123456789	John	Smith	888665555	2
333445555	Franklin	Wong	888665555	2
453453453	Joyce	English	333445555	3
666884444	Ramesh	Narayan	333445555	3
987654321	Jennifer	Wallace	333445555	3
999887777	Alicia	Zelaya	333445555	3
987987987	Ahmed	Jabbar	987654321	4

8 rows in set (0.04 sec)

The **CASE** statement in **SQL** is a versatile conditional expression that enables us to incorporate **conditional logic** directly within our queries. It allows you to return specific results based on certain conditions, enabling dynamic query outputs.

- It is commonly used to create new columns based on conditional logic, provide custom values, or control query outputs based on certain conditions.
- If no condition is true then the ELSE part will be executed. If there is no ELSE part then it returns NULL.

Syntax:

```
CASE  
WHEN condition THEN result1  
WHEN condition THEN result2  
...  
Else result  
END
```

```
CASE expression  
    WHEN value1 THEN result1  
    WHEN value2 THEN result2  
    ...  
    ELSE default_result  
END
```

Write an SQL query to display the first name and salary of each employee, along with a derived column that categorizes employees into 'High', 'Medium', or 'Low' salary levels based on the following conditions:

- Salary $\geq$ 55000 THEN 'High'
- Salary $\geq$ 40,000 THEN 'Medium' else low

Query: SELECT Fname, Salary,

CASE

WHEN Salary $\geq$ 55000 THEN 'High'

WHEN Salary $\geq$ 40000 THEN 'Medium'

ELSE 'Low'

END AS Salary_Level FROM employee;

Fname	Salary	Salary_Level
John	30000.00	Low
Franklin	40000.00	Medium
Joyce	25000.00	Low
Ramesh	38000.00	Low
James	55000.00	High
Jennifer	43000.00	Medium
Ahmed	25000.00	Low
Alicia	25000.00	Low

8 rows in set (0.00 sec)

Write an SQL query to display the first name and salary of each employee, along with a new column called Bonus_Status that categorizes employees based on their salary as follows:

- Salary $\geq 50,000 \rightarrow$ 'Eligible for 20% Bonus'
- Salary between 40,000 and 49,999 $\rightarrow$ 'Eligible for 10% Bonus'
- Salary $< 40,000 \rightarrow$ 'No Bonus'

—TRY this

Expected Output:

Fname	Salary	Bonus_Status
John	30000.00	No Bonus
Franklin	40000.00	Eligible for 10% Bonus
Joyce	25000.00	No Bonus
Ramesh	38000.00	No Bonus
James	55000.00	Eligible for 20% Bonus
Jennifer	43000.00	Eligible for 10% Bonus
Ahmed	25000.00	No Bonus
Alicia	25000.00	No Bonus

Solution for the previous query

```
SELECT
```

```
    Fname, Salary,
```

```
    CASE
```

```
        WHEN Salary >= 50000 THEN 'Eligible for 20% Bonus'
```

```
        WHEN Salary >= 40000 THEN 'Eligible for 10% Bonus'
```

```
        ELSE 'No Bonus'
```

```
    END AS Bonus_Status
```

```
FROM employee;
```

Write a query to display each employee's first name, number of projects they are working on, and a new column Workload_Status as follows:

- 3 or more projects → Overloaded
- 2 projects → Normal Load
- 1 or 0 projects → Light Load

Expected Output:

Fname	Project_Count	Workload_Status
John	2	Normal Load
Franklin	4	Overloaded
Joyce	2	Normal Load
Ramesh	1	Light Load
James	1	Light Load
Jennifer	2	Normal Load
Ahmed	2	Normal Load
Alicia	2	Normal Load

Solution for the previous query

```
SELECT
    e.Fname, COUNT(w.Pno) AS Project_Count,
    CASE
        WHEN COUNT(w.Pno) >= 3 THEN 'Overloaded'
        WHEN COUNT(w.Pno) = 2 THEN 'Normal Load'
        ELSE 'Light Load'
    END AS Workload_Status
FROM employee e
LEFT JOIN works_on w ON e.Ssn = w.Essn
GROUP BY e.Fname;
```

The same query using a CTE to show how CASE can be combined with CTE for better readability

```
WITH Project_Counts AS (  
    SELECT e.Fname, COUNT(w.Pno) AS Proj_Count  
    FROM employee e LEFT JOIN works_on w ON e.Ssn = w.Essn  
    GROUP BY e.Fname  
)  
SELECT Fname, Proj_Count,  
    CASE  
        WHEN Proj_Count >= 3 THEN 'Overloaded'  
        WHEN Proj_Count = 2 THEN 'Normal Load'  
        ELSE 'Light Load'  
    END AS Workload_Status  
FROM Project_Counts;
```

- 1) Which SQL clause is used to define a Common Table Expression (CTE)?
A) CREATE B) WITH C) TEMP D) SET

- 2) A Common Table Expression (CTE) provides a _____ result set that is defined and used strictly within the execution scope of a single SQL statement.

- 3) When creating a recursive CTE, which operator is used to combine the results of the initial base case query with the results of the recursive query?
A) JOIN
B) INTERSECT
C) UNION ALL
D) EXCEPT

4) A recursive CTE consists of two main components: the recursive member, which references the CTE itself, and the _____ member, which forms the initial base case of the recursion.

5) When writing a CASE expression in SQL, what happens if none of the specified WHEN conditions evaluate to true, and no ELSE part is provided?

- A) The query throws a syntax error.
- B) It returns FALSE.
- C) It returns 0.
- D) It returns NULL.

6) If you need to define multiple CTEs for use within a single main query, you can do so by separating each CTE definition with _____.

Answers:

1. B
2. temporary
3. C
4. anchor
5. D
6. commas



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Window Functions

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

What are window functions?

In MySQL, a window function performs a calculation across a set of rows that are related to the current row, without collapsing rows into a single output (unlike GROUP BY).

Window functions let you do calculations across rows, but instead of grouping them like SUM() or AVG() normally does, they let you keep every row and just add extra calculated values.

How are they different from aggregate functions?

While aggregate functions collapse rows into one result, window functions add new values without collapsing rows.

Database Management Systems

Windows Functions in SQL - Example

Sample schema - Departments and Employees table

```
[mysql> select * from departments;
```

dept_id	dept_name
1	HR
2	Sales
3	Engineering

3 rows in set (0.01 sec)

```
[mysql> select * from employees;
```

emp_id	emp_name	department	salary	hire_date
101	Alice	Sales	5000	2020-01-15
102	Bob	Sales	7000	2019-03-12
103	Charlie	Engineering	9000	2018-06-01
104	David	Engineering	8000	2021-07-19
105	Eva	HR	6000	2020-09-30
106	Frank	Sales	7500	2022-02-11

6 rows in set (0.00 sec)

Database Management Systems

Windows Functions in SQL - Example

```
mysql> SELECT department, AVG(salary)
-> FROM employees
-> GROUP BY department;
```

department	AVG(salary)
Sales	6500.0000
Engineering	8500.0000
HR	6000.0000

3 rows in set (0.00 sec)

On executing the query shown using aggregate function AVG we get a single row for each department.

```
mysql> SELECT
-> emp_name, department, salary,
-> AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
-> FROM employees;
```

emp_name	department	salary	dept_avg_salary
Charlie	Engineering	9000	8500.0000
David	Engineering	8000	8500.0000
Eva	HR	6000	6000.0000
Alice	Sales	5000	6500.0000
Bob	Sales	7000	6500.0000
Frank	Sales	7500	6500.0000

6 rows in set (0.00 sec)

Using window functions, all rows in the employee table are maintained along with average salary of the department they belong to.

Why are window functions required?

Window functions let you analyze data across rows without losing individual rows. They're required when you want both:

1. Detailed row-level data (every employee, every order, etc.)
2. Aggregates or comparisons across groups (totals, averages, rankings, etc.)

Traditional GROUP BY only gives you summary rows, but window functions give you summary + detail in the same query.

Common use cases of window functions include Ranking & Row Numbers, Running Totals & Cumulative Sums, Moving Averages (Trends), Comparisons to Group Values, Percentiles & Distribution, Lag & Lead (Previous/Next Row Access) etc.

Database Management Systems

Windows Functions in SQL - Types



Types of Window Functions in SQL

- **Aggregate functions with OVER()**
SUM(), AVG(), MIN(), MAX(), COUNT()
→ applied over a partition/window instead of collapsing into one row.
- **Ranking functions**
Some of the ranking functions are ROW_NUMBER() , RANK(), DENSE_RANK()
- **Value functions**
Some of the functions are LEAD(), LAG(), FIRST_VALUE() LAST_VALUE()

Syntax

```
SELECT column_name1, column_name2....  
window_function(column_name2)  
OVER([PARTITION BY column_name1]) AS new_column  
FROM table_name;
```

Key Terms

window_function= any aggregate or ranking function

column_name1 and column_name2 = columns to be selected

column_name2= column on which window function is to be applied

new_column= Name of new column

table_name= Name of table

Database Management Systems

Windows Functions in SQL - Example



Let's now try to understand what this query does in detail:

```
SELECT name, department, salary,AVG(salary)  
OVER (PARTITION BY department)  
AS dept_avg_salary  
FROM employees;
```

1. OVER clause tells SQL to not collapse rows like GROUP BY and instead, calculate an aggregate in a 'window' while keeping all rows visible.
2. PARTITION BY department creates separate "mini-groups" (windows) for each department i.e one window for Sales,one window for Engineering,one window for HR. Within each window, calculations are done.
3. AVG(salary)
Inside each department, SQL calculates the average salary.

Database Management Systems

Aggregate Windows Functions in SQL



1. Aggregate Window Function

Aggregate window functions calculate aggregates over a window of rows while retaining individual rows.

Common aggregate functions include:

SUM(): Sums values within a window.

AVG(): Calculates the average value within a window.

COUNT(): Counts the rows within a window.

MAX(): Returns the maximum value in the window.

MIN(): Returns the minimum value in the window.

Database Management Systems

Aggregate Windows Functions in SQL

Some other examples:

1. To find Maximum salary in each department:

```
SELECT emp_ID, emp_name, Department, Salary, MAX(Salary)
OVER (PARTITION BY Department) AS MaxDeptSalary
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary, MAX(Salary)
[    -> OVER (PARTITION BY Department) AS MaxDeptSalary FROM Employees;
+-----+-----+-----+-----+-----+
| Emp_ID | emp_name | Department | Salary | MaxDeptSalary |
+-----+-----+-----+-----+-----+
| 103 | Charlie | Engineering | 9000 | 9000 |
| 104 | David | Engineering | 8000 | 9000 |
| 105 | Eva | HR | 6000 | 6000 |
| 101 | Alice | Sales | 5000 | 7500 |
| 102 | Bob | Sales | 7000 | 7500 |
| 106 | Frank | Sales | 7500 | 7500 |
+-----+-----+-----+-----+-----+
6 rows in set (0.04 sec)
```

Database Management Systems

Aggregate Windows Functions in SQL

2. Count of employees in each department:

```
SELECT Emp_ID, emp_name, Department, Salary, COUNT(*)  
OVER (PARTITION BY Department) AS DeptEmployeeCount  
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary,  
-> COUNT(*) OVER (PARTITION BY Department) AS DeptEmployeeCount  
-> FROM Employees;
```

Emp_ID	emp_name	Department	Salary	DeptEmployeeCount
103	Charlie	Engineering	9000	2
104	David	Engineering	8000	2
105	Eva	HR	6000	1
101	Alice	Sales	5000	3
102	Bob	Sales	7000	3
106	Frank	Sales	7500	3

6 rows in set (0.03 sec)

2. Ranking Window Function

Ranking window functions assign a position (rank/number) to each row **within a partition**, usually based on an ORDER BY clause.

Common Ranking Functions:

- 1) **RANK()** : Assigns ranks to rows with the same value receiving the same rank. If there are ties, the next rank is skipped.
- 2) **DENSE_RANK()** : Similar to RANK(), but ranks are not skipped. If there are ties, the next rank is consecutive.
- 3) **ROW_NUMBER()** : Assigns a unique sequential number to each row (1, 2, 3 ...). No ties, every row has a different number.

Database Management Systems

Ranking Window Functions in SQL



1. RANK() Function

- RANK() is a window function that assigns a rank (1st, 2nd, 3rd, ...) to each row within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank, and the next rank is skipped. For example: if two rows are tied at rank 1, the next row will be rank 3 (not 2) as shown below.
- **Example:** select emp_name, department, salary, RANK() OVER (PARTITION by department ORDER BY salary DESC) AS rank_in_dept FROM Employees;

```
[mysql> SELECT emp_name, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept FROM Employees;
```

emp_name	department	salary	rank_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	1
Alice	Sales	5000	3

```
6 rows in set (0.01 sec)
```

Database Management Systems

Ranking Window Functions in SQL

2. DENSE_RANK() Function

- DENSE_RANK() is a window function that also assigns ranks (1st, 2nd, 3rd, ...) to rows within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank.
- No ranks are skipped, Alice is now numbered 2 instead of 3 when rank was used.
- Example: select emp_name, department, salary, DENSE_RANK() OVER (PARTITION by department ORDER BY salary DESC) AS dense_rank_in_dept FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
-> DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rank_in_dept  
-> FROM Employees;
```

emp_name	department	salary	dense_rank_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	1
Alice	Sales	5000	2

6 rows in set (0.01 sec)

Database Management Systems

Ranking Window Functions in SQL

3. ROW_NUMBER() Function

- The ROW_NUMBER() function assigns a unique sequential number to each row within a partition. The numbering starts at 1 for each partition.
- Unlike RANK() or DENSE_RANK(), there are no ties → every row gets its own distinct number, even if values are equal.
- Example: select emp_name, department, salary, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as row_num_in_dept from Employees;

```
mysql> SELECT emp_name, department, salary,  
-> ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num_in_dept  
-> FROM Employees;
```

emp_name	department	salary	row_num_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	2
Alice	Sales	5000	3

6 rows in set (0.04 sec)

1: LAG() Function: This function lets you look at the value of a column from a previous row within the same partition.

- It is very useful for comparing current row values with earlier ones (like salary growth, stock price change, etc.).
- You can specify how many rows back you want to look (default = 1 row).

General syntax:

`LAG(column_name, offset, default_value)`

`OVER (PARTITION BY partition_column`

`ORDER BY order_column)`

`column_name` is the column you want to peek ahead into.

`offset` is how many rows forward to look (default = 1).

`default_value` is optional, what to return if there's no row that far ahead (default = NULL).

Example: SELECT emp_name, department, salary, LAG(salary,1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
->      LAG(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary  
-> FROM Employees;
```

emp_name	department	salary	prev_salary
Charlie	Engineering	9000	NULL
David	Engineering	8000	9000
Eva	HR	6000	NULL
Bob	Sales	7000	NULL
Frank	Sales	7000	7000
Alice	Sales	5000	7000

6 rows in set (0.05 sec)

Use Cases of LAG():

- Compare current salary with previous salary in the same department.
- Track daily stock price changes by comparing today vs yesterday.
- Calculate differences, growth %, or trends row by row.

2 LEAD() Function

- The LEAD() function lets you look at the value of a column from a future row within the same partition. It is the opposite of LAG().
- Useful for comparing the current row with the *next one* (e.g., predicting trends, seeing who comes after).

General syntax:

```
LEAD(column_name, offset, default_value)  
OVER (PARTITION BY partition_column  
ORDER BY order_column)
```

column_name is the column you want to peek ahead into.

offset is how many rows forward to look (default = 1).

default_value is optional, what to return if there's no row that far ahead (default = NULL).

Example: SELECT emp_name, department, salary, LAG(salary,1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
->      LEAD(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS next_salary  
-> FROM Employees;
```

emp_name	department	salary	next_salary
Charlie	Engineering	9000	8000
David	Engineering	8000	NULL
Eva	HR	6000	NULL
Bob	Sales	7000	7000
Frank	Sales	7000	5000
Alice	Sales	5000	NULL

6 rows in set (0.04 sec)

Use Cases of LEAD():

- See what the next employee's salary is in the same department.
- Calculate salary differences ahead of time (LEAD(salary) - salary).
- Compare today's stock price with tomorrow's price (useful in analytics & predictions).
- Identify transitions (e.g., who replaces whom in a sequence).

Practical Use Cases for Window Functions

Window functions are extremely versatile and can be used in a variety of practical scenarios. Below are some examples of how these functions can be applied in real-world data analysis.

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount
1	101	Alice	HR	C001	2023-01-05	Jan	1000
2	101	Alice	HR	C002	2023-02-10	Feb	2000
3	101	Alice	HR	C001	2023-03-15	Mar	1500
4	102	Bob	HR	C003	2023-01-20	Jan	1200
5	102	Bob	HR	C001	2023-02-25	Feb	1800
6	103	Charlie	IT	C004	2023-01-10	Jan	7000
7	103	Charlie	IT	C005	2023-02-18	Feb	7500
8	104	David	IT	C006	2023-01-28	Jan	7200

Practical Use Cases for Window Functions

SQL statements to create table and populate the values:

```
CREATE TABLE Sales (SaleID INT PRIMARY KEY, EmpID INT, EmpName VARCHAR(50),  
Department VARCHAR(50), CustomerID VARCHAR(10), SaleDate DATE, Month VARCHAR(10),  
Amount DECIMAL(10,2));
```

```
INSERT INTO Sales (SaleID, EmpID, EmpName, Department, CustomerID, SaleDate, Month,  
Amount)
```

VALUES

```
(1, 101, 'Alice', 'HR', 'C001', '2023-01-05', 'Jan', 1000),  
(2, 101, 'Alice', 'HR', 'C002', '2023-02-10', 'Feb', 2000),  
(3, 101, 'Alice', 'HR', 'C001', '2023-03-15', 'Mar', 1500),  
(4, 102, 'Bob', 'HR', 'C003', '2023-01-20', 'Jan', 1200),  
(5, 102, 'Bob', 'HR', 'C001', '2023-02-25', 'Feb', 1800),  
(6, 103, 'Charlie', 'IT', 'C004', '2023-01-10', 'Jan', 7000),  
(7, 103, 'Charlie', 'IT', 'C005', '2023-02-18', 'Feb', 7500),  
(8, 104, 'David', 'IT', 'C006', '2023-01-28', 'Jan', 7200);
```

1. Retrieve top 2 Sales by each Employee and rank them.

SELECT * FROM (SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount DESC) AS SaleRank FROM Sales) AS ranked WHERE SaleRank <= 2;

```
[mysql> SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount DESC) AS SaleRank FROM Sales ) AS ranked WHERE SaleRank <= 2;
```

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount	SaleRank
2	101	Alice	HR	C002	2023-02-10	Feb	2000	1
3	101	Alice	HR	C001	2023-03-15	Mar	1500	2
5	102	Bob	HR	C001	2023-02-25	Feb	1800	1
4	102	Bob	HR	C003	2023-01-20	Jan	1200	2
7	103	Charlie	IT	C005	2023-02-18	Feb	7500	1
6	103	Charlie	IT	C004	2023-01-10	Jan	7000	2
8	104	David	IT	C006	2023-01-28	Jan	7200	1

```
7 rows in set (0.01 sec)
```

2. Running Total of Sales per Employee

SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER
(PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;
```

EmpID	EmpName	Month	Amount	RunningTotal
101	Alice	Jan	1000	1000
101	Alice	Feb	2000	3000
101	Alice	Mar	1500	4500
102	Bob	Jan	1200	1200
102	Bob	Feb	1800	3000
103	Charlie	Jan	7000	7000
103	Charlie	Feb	7500	14500
104	David	Jan	7200	7200

```
8 rows in set (0.00 sec)
```


3. First Sale of Each Customer

SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales) AS first_sales WHERE rn = 1;

```
[mysql> SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales ) AS first_sales WHERE rn = 1;
```

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount	rn
1	101	Alice	HR	C001	2023-01-05	Jan	1000	1
2	101	Alice	HR	C002	2023-02-10	Feb	2000	1
4	102	Bob	HR	C003	2023-01-20	Jan	1200	1
6	103	Charlie	IT	C004	2023-01-10	Jan	7000	1
7	103	Charlie	IT	C005	2023-02-18	Feb	7500	1
8	104	David	IT	C006	2023-01-28	Jan	7200	1

```
6 rows in set (0.01 sec)
```


4. Compare Current Sale with Previous Sale (LAG)

SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;
```

EmpID	EmpName	Month	Amount	PrevAmount
101	Alice	Jan	1000	NULL
101	Alice	Feb	2000	1000
101	Alice	Mar	1500	2000
102	Bob	Jan	1200	NULL
102	Bob	Feb	1800	1200
103	Charlie	Jan	7000	NULL
103	Charlie	Feb	7500	7000
104	David	Jan	7200	NULL

```
8 rows in set (0.00 sec)
```

5. Rank Employees by Sale Amount within Department

SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;
```

EmpID	EmpName	Department	Amount	RankInDept
101	Alice	HR	2000	1
102	Bob	HR	1800	2
101	Alice	HR	1500	3
102	Bob	HR	1200	4
101	Alice	HR	1000	5
103	Charlie	IT	7500	1
104	David	IT	7200	2
103	Charlie	IT	7000	3

```
8 rows in set (0.00 sec)
```

- 1) How do window functions fundamentally differ from traditional aggregate functions used with GROUP BY?
 - A) Window functions completely remove duplicate data from the output.
 - B) Window functions add calculated values without collapsing individual rows into a single result.
 - C) Window functions can only be used with string data types.
 - D) Window functions calculate aggregates for the entire table at once, ignoring partitions.

- 2) In the syntax of a window function, the _____ clause tells SQL to create separate "mini-groups" or windows (such as grouping by department) before performing calculations.

- 3) Which ranking window function assigns the same rank to tied values but ensures that the next rank in the sequence is not skipped?
 - A) RANK()
 - B) ROW_NUMBER()
 - C) DENSE_RANK()
 - D) NTILE()

- 4) The _____() function assigns a unique, sequential number to each row within a partition, meaning no two rows will ever get the same number even if their values are tied.
- 5) If you need to compare today's stock price with yesterday's price by looking at the value from a previous row within the same partition, which function should you use?
- A) LEAD()
 - B) LAG()
 - C) PREVIOUS()
 - D) FIRST_VALUE()
- 6) When using the LEAD() or LAG() functions, you can specify an offset to determine how many rows forward or backward to look. What is the default offset if you do not explicitly provide one?
- A) 0
 - B) 2
 - C) NULL
 - D) 1

Answers:

1. B
2. PARTITION BY
3. C
4. ROW_NUMBER
5. B
6. D

- 1) MySQL Window Functions Reference
- 2) SQL Server Window Functions (Microsoft Docs)
- 3) Geeks for Geeks



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Full Text Search in MySQL

Anchor:

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL in Database Design

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering



Full-Text Search is a feature in MySQL where you can search for words or phrases in large text-based columns (CHAR, VARCHAR, TEXT) using natural language processing (NLP). Instead of just checking if a string is present, full-text search **looks for relevant matches** and **ranks results based on how well they match the search terms**.

Full-Text Search in MySQL allows you to perform advanced text-based searches on columns with textual data. It is particularly useful for searching large text fields like articles, descriptions, or comments. It is more powerful than **LIKE/REGEXP**, since it ranks results by relevance.

Features -

- **Full-Text Indexes** → much faster than **LIKE**
- **Search Modes** → Natural Language, Boolean, Query Expansion
- **Relevance Ranking** → Scores based on closeness of match
- Handles stopwords, synonyms, and word proximity.

The key feature of Full Text Search is that it uses the **MATCH(column) AGAINST('search terms')** syntax to search.

Suppose you have a table called 'Product' with different columns such as product_id, product_name, description and price. Suppose you want to search for the products that have "smartphone" in their name or description. The SQL query for that would be :

```
SELECT * FROM products  
WHERE MATCH(product_name, description)  
AGAINST('smartphone' IN BOOLEAN MODE);
```

Here,

MATCH function is used to perform full text search on product_name and description column.

AGAINST specifies the search term "smartphone" and in boolean mode.

Database Management Systems

Full Text Search vs Normal String Search



Feature	LIKE/REGEXP	FULL TEXT SEARCH
Index Usage	Does not use indexes effectively. Slow for large data	Uses special indexes Faster searches
Relevance Ranking	No relevance ranking, just returns matches	Results ranked by relevance
Natural Language Support	Not supported	Supported (Natural Language Mode)
Word Boundary Detection	Not accurate (matches substrings like "cat" in "concatenate")	Accurate word boundary detection
Boolean Logic	Not supported	Supported (AND, OR, NOT operators)
Phrase Searching	Possible but imprecise	Supported and more accurate

By default or with the **IN NATURAL LANGUAGE MODE** modifier, the [MATCH\(\)](#) function performs a natural language search for a string against a text collection.

A collection is a set of one or more columns included in a FULLTEXT index. The search string is given as the argument to **AGAINST()**.

For each row in the table, [MATCH\(\)](#) returns a relevance value; that is, a similarity measure between the search string and the text in that row in the columns named in the [MATCH\(\)](#) list.

Working :

1. Use columns of type VARCHAR or TEXT
2. Create a **FULLTEXT INDEX** on the column(s)
3. Use MATCH(column1, column2) with AGAINST('word')

Example :

```
CREATE TABLE posts (  
    id INT UNSIGNED AUTO_INCREMENT NOT NULL PRIMARY KEY,  
    title VARCHAR(200),  
    body TEXT  
) ENGINE=InnoDB;
```

```
ALTER TABLE posts  
ADD FULLTEXT(title, body); -- To enable FTS
```

```
INSERT INTO posts (title, body) VALUES  
( 'MySQL Tutorial', 'Learn the basics of database systems'),  
( 'Advanced Database', 'We explore indexing and performance'),  
( 'Cooking Tips', 'Always use fresh ingredients'),  
( 'MySQL vs PostgreSQL', 'Comparison of two database systems'),  
( 'Oracle Tips', 'Learn how to use Oracle effectively');
```

Query : Select those rows where either the body or the title have the word 'database'

***SELECT * FROM posts
WHERE MATCH (title, body)
AGAINST ('database' IN NATURAL LANGUAGE MODE);***

```
mysql> SELECT * from posts
-> WHERE MATCH (title, body)
-> AGAINST ('database' IN NATURAL LANGUAGE MODE);
+----+-----+-----+
| id | title                | body                                     |
+----+-----+-----+
| 1  | MySQL Tutorial       | Learn the basics of database systems |
| 2  | Advanced Database    | We explore indexing and performance  |
| 4  | MySQL vs PostgreSQL  | Comparison of two database systems   |
+----+-----+-----+
3 rows in set (0.00 sec)
```

Explanation :

- We perform a **full-text search** in **natural language mode** for the word "database".
- It searches the title and body fields for relevance to the term "database".
- Returns rows where the relevance score is **non-zero** (i.e., the word "database" appears and is significant).

Natural Language Mode :

1. Uses natural language processing to rank results based on relevance.
2. Ignores **stopwords** (like "a", "the", etc.) and very short words (less than 4 characters, by default).
3. **Does not require special syntax** (unlike BOOLEAN mode).

Mode	Description
Natural Language Mode	Default mode; ranks relevance
Boolean Mode	Allows operators (+, -, *) for complex queries

In MySQL Full-Text Search, **Relevance** is a numeric score that tells **how closely a row matches the search terms**.

It's a **floating-point number** (0.0, 0.25, 1.2, etc.)

Higher Score = Better Match

Relevance is calculated using:

1. How often the search term appears in the column
2. How rare the term is across all rows.
3. Length of the document (shorter docs = higher relevance if term is present)

Example :

```
SELECT id, title,  
       MATCH(title, body) AGAINST('database') AS relevance  
FROM posts  
WHERE MATCH(title, body) AGAINST('database')  
ORDER BY relevance DESC;
```

What it does:

1. Selects matching rows which match against the word 'database'
2. Shows the relevance score
3. Orders results by relevance descending (highest match first)

Now: What If You Don't Mention Relevance?

SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database');

What happens:

Results are still matched using full-text search.

MySQL will still use the **FULLTEXT** index.

By default, **MySQL** will sort the results by relevance, if and only if:

1. You **don't** use any **ORDER BY**.
2. You're using **MATCH(...) AGAINST(...)** in the **WHERE** clause.
3. The **FULLTEXT** index is being used (not bypassed).
4. It's a **simple query** (no joins).

So yes — even **without ORDER BY** or showing relevance, the rows will still be sorted by **highest relevance, automatically**, under those conditions.

Important Note: If you **add any other ORDER BY clause**, MySQL will stop auto-sorting by relevance.

```
SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database')  
ORDER BY id; -- Will sort by id, not relevance
```

What is BOOLEAN MODE?

BOOLEAN MODE allows you to:

1. Use **operators** (+, -, ", *, etc.) to include, exclude, or group terms.
2. Search for **partial words** using wildcards (*).
3. Write **complex search logic** (AND, OR, NOT-like behavior).
4. Search even if the word is a **stopword** or appears in **every row**.

You must **explicitly write**:

AGAINST ('your_query' IN BOOLEAN MODE)

Syntax Example

```
SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -database' IN  
BOOLEAN MODE);
```

-- Here, we are including 'mysql' and excluding 'database';

Database Management Systems

Boolean Mode

Some Common Boolean Operators

Operator	Meaning	Example
+	Word must be present	+mysql (must contain "mysql")
-	Word must not be present	-database (must not contain "database")
(none)	Word is optional, but affects relevance	database-row with the given word are ranked higher
""	Search exact phrase	"mysql tutorial"
*	Wildcard (suffix only)	data* (matches "data", "database")
()	Group terms	+(mysql database) –must contain mysql or database

Example Queries in Boolean Mode

1. Include one word, exclude another

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -oracle' IN BOOLEAN MODE);

Must contain "mysql", must **not** contain "oracle"

2. Exact Phrase Match

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('"database systems"' IN BOOLEAN MODE);

Must contain the **exact phrase** 'database systems'

3. Prefix Search with Wildcard

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('data*' IN BOOLEAN MODE);

Matches: "data", "database", "databases", etc.

Only suffix wildcards are supported (e.g., data*), not *data.

4. Grouping Words

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+(mysql database) -oracle' IN BOOLEAN MODE);

Database Management Systems

Boolean Mode - Practical Exercise



Here's a practical exercise on **Full-Text Search in MySQL** using Boolean and Natural Language Modes. It has :

- A sample table 'articles'
- Inserted data
- Practice questions ranging from basic to advanced

CREATE TABLE articles (article_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255), content TEXT, FULLTEXT (title, content));

INSERT INTO articles (title, content) VALUES

('Introduction to Databases', 'Databases are organized collections of data. MySQL is a popular open-source database.'),

('MySQL Optimization Techniques', 'Learn indexing, query plans, and caching to speed up MySQL queries.'),

('Oracle vs MySQL', 'Comparison of Oracle and MySQL databases based on performance and cost.'),

('PostgreSQL Guide', 'PostgreSQL is an advanced open-source RDBMS.'),

('Data Science and SQL', 'SQL plays a key role in data science workflows.'),

('MySQL Tutorial for Beginners', 'This MySQL tutorial covers basic to intermediate topics for new developers.'),

('Cooking for Beginners', 'This guide is about basic cooking techniques and not related to databases.');

Database Management Systems

Boolean Mode - Practical Exercise



1. Retrieve all articles that mention the word “**MySQL**” using natural language mode.
1. Get the relevance score for the term "**MySQL**".
1. Find articles that **must include** "**MySQL**" but **must not include** "**Oracle**".
1. Find articles that contain **either** "**tutorial**" or "**guide**".
1. Find articles that **must contain** “**data**” and any word that starts with "**scien**" (like "science", "scientific").

(Refer notes for solutions!)

- 1) Which of the following is a key advantage of using Full-Text Search over standard string matching functions like LIKE or REGEXP?
 - A) It uses standard indexes instead of special full-text indexes.
 - B) It returns exact substring matches without word boundary detection.
 - C) It ranks the results based on relevance.
 - D) It only works on numeric data types.

- 2) The core syntax of Full-Text Search in MySQL involves using the _____ function to specify the columns being searched, combined with the AGAINST() function to specify the search terms.

- 3) When calculating a relevance score for a search result, which of the following factors does MySQL consider?
 - A) The alphabetical order of the words.
 - B) How rare the term is across all rows in the table.
 - C) The exact character count of the entire database.
 - D) The number of tables joined in the query.

4) If you do not specify a mode, MySQL will default to using _____ Mode, which utilizes natural language processing and automatically ignores common stopwords.

5) When using Boolean Mode, which operator would you place immediately before a word to guarantee that the word must be present in the returned rows?

A) - B) * C) + D) “ “

6) Fill in the blank: MySQL will automatically sort your search results by highest relevance score by default, as long as you do not explicitly include an _____ clause in your query.

Answers:

1. C
2. MATCH
3. B
4. Natural Language
5. C
6. ORDER BY



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu



Database Management Systems

Common Table Expressions - CTE

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof Mahitha G
Department of Computer Science and Engineering

Recursive CTEs

- A recursive CTE is one that references itself within that CTE.
- The recursive **CTE** is useful when working with hierarchical data as the CTE continues to execute until the query returns the entire hierarchy.
- A typical example of hierarchical data is a table that includes a list of **employees** like in the company database employee has recursive relationship, which represents the employee–supervisor relationship.
- In SQL, recursive queries are typically implemented using Common Table Expressions (CTEs) with the WITH RECURSIVE clause.

Here's the general syntax for creating a recursive query

```
WITH RECURSIVE cte_name (column_list) AS (
```

```
  -- Anchor member: The initial query that forms the base case
```

```
  SELECT ...
```

```
  FROM ...
```

```
  WHERE ...
```

```
  UNION ALL
```

```
  -- Recursive member: The query that refers to the CTE itself
```

```
  SELECT ...
```

```
  FROM cte_name -- Refer to the CTE within itself
```

```
  WHERE ...
```

```
)-- The final SELECT statement outside the CTE to retrieve the result
```

```
SELECT ...FROM cte_name;
```

Here's an explanation of each part of the syntax:

- **cte_name:** This is the name you give to the Common Table Expression (CTE), which you can use to refer to it within the CTE itself and in the final SELECT statement.
- **column_list:** Specify the list of columns you want to include in the CTE's result set.
- **Anchor member:** This is the initial query that forms the base case of the recursion. It selects the starting point or points for the recursive process. It's important to ensure that this query doesn't reference the CTE itself.

Here's an explanation of each part of the syntax:

- **UNION ALL:** This is used to combine the results of the anchor member with the results of the recursive member. UNION ALL is used because it includes duplicate rows, which is often necessary in recursive queries.
- **Recursive member:** This is the query that refers to the CTE itself, creating the recursion. It selects records based on relationships established in previous iterations. This query must reference the CTE by its name and should include a termination condition to stop the recursion.
- **Final SELECT statement:** This query retrieves the final result set from the CTE. It typically follows the recursive part of the CTE definition.

Example: Given an example of a recursive query that counts down from 5 to 1:

```
WITH RECURSIVE Countdown (n) AS (
```

```
-- Anchor member
```

```
SELECT 5
```

```
UNION ALL
```

```
-- Recursive member
```

```
SELECT n - 1
```

```
FROM Countdown
```

```
WHERE n > 1
```

```
)
```

```
-- Final SELECT statement
```

```
SELECT n FROM Countdown;
```

n
5
4
3
2
1

Example: Given an example of a recursive query that counts down from 1 to 5:

```
WITH RECURSIVE cte(n) AS (  
    -- Anchor part of the query  
    SELECT 1  
    UNION ALL  
    -- Recursive part of the query  
    SELECT n + 1 FROM cte WHERE n < 5  
)  
SELECT * FROM cte;
```

n
1
2
3
4
5

Execution Flow:

A **recursive CTE** has two parts:

1. **Anchor member** → the initial result set
2. **Recursive member** → repeatedly applied until it stops producing rows

The database engine runs it in an iterative fashion.

Run the anchor query first: `SELECT 1`

This produces the first row: 1.

Then, Apply the recursive part - SQL substitutes the current contents of cte into:

```
SELECT n + 1 FROM cte WHERE n < 5
```

From (1), since $1 < 5$, it generates: 2 which is added to the result.

Give a query to extract the organizational hierarchy beginning with a specific employee, typically the CEO, and extending downward through the reporting structure.

```
with recursive orghierarchy as(
  select ssn,fname,lname,super_ssn, 1 as level
  from employee
  where super_ssn is null
  union all
  select e.ssn,e.fname,e.lname,e.super_ssn, OH.level+1
  from employee e
  join orghierarchy OH on e.super_ssn=OH.ssn
)
select * from orghierarchy;
```

Output:

ssn	fname	lname	super_ssn	level
888665555	James	Borg	NULL	1
123456789	John	Smith	888665555	2
333445555	Franklin	Wong	888665555	2
453453453	Joyce	English	333445555	3
666884444	Ramesh	Narayan	333445555	3
987654321	Jennifer	Wallace	333445555	3
999887777	Alicia	Zelaya	333445555	3
987987987	Ahmed	Jabbar	987654321	4

8 rows in set (0.04 sec)

The **CASE** statement in **SQL** is a versatile conditional expression that enables us to incorporate **conditional logic** directly within our queries. It allows you to return specific results based on certain conditions, enabling dynamic query outputs.

- It is commonly used to create new columns based on conditional logic, provide custom values, or control query outputs based on certain conditions.
- If no condition is true then the ELSE part will be executed. If there is no ELSE part then it returns NULL.

Syntax:

```
CASE case_value  
WHEN condition THEN result1  
WHEN condition THEN result2  
...  
Else result  
END CASE;
```

Write an SQL query to display the first name and salary of each employee, along with a derived column that categorizes employees into 'High', 'Medium', or 'Low' salary levels based on the following conditions:

- Salary $\geq$ 55000 THEN 'High'
- Salary $\geq$ 40,000 THEN 'Medium' else low

Query: SELECT Fname, Salary,

CASE

WHEN Salary $\geq$ 55000 THEN 'High'

WHEN Salary $\geq$ 40000 THEN 'Medium'

ELSE 'Low'

END AS Salary_Level

FROM employee;

Fname	Salary	Salary_Level
John	30000.00	Low
Franklin	40000.00	Medium
Joyce	25000.00	Low
Ramesh	38000.00	Low
James	55000.00	High
Jennifer	43000.00	Medium
Ahmed	25000.00	Low
Alicia	25000.00	Low

8 rows in set (0.00 sec)

Write an SQL query to display the first name and salary of each employee, along with a new column called Bonus_Status that categorizes employees based on their salary as follows:

- Salary $\geq 50,000 \rightarrow$ 'Eligible for 20% Bonus'
- Salary between 40,000 and 49,999 $\rightarrow$ 'Eligible for 10% Bonus'
- Salary $< 40,000 \rightarrow$ 'No Bonus'

Expected Output:

—TRY this

Fname	Salary	Bonus_Status
John	30000.00	No Bonus
Franklin	40000.00	Eligible for 10% Bonus
Joyce	25000.00	No Bonus
Ramesh	38000.00	No Bonus
James	55000.00	Eligible for 20% Bonus
Jennifer	43000.00	Eligible for 10% Bonus
Ahmed	25000.00	No Bonus
Alicia	25000.00	No Bonus

Solution for the previous query

SELECT

Fname, Salary,

CASE

WHEN Salary >= 50000 THEN 'Eligible for 20% Bonus'

WHEN Salary >= 40000 THEN 'Eligible for 10% Bonus'

ELSE 'No Bonus'

END AS Bonus_Status

FROM employee;

Write a query to display each employee's first name, number of projects they are working on, and a new column Workload_Status as follows:

- 3 or more projects → Overloaded
- 2 projects → Normal Load
- 1 or 0 projects → Light Load

Expected Output:

Fname	Project_Count	Workload_Status
John	2	Normal Load
Franklin	4	Overloaded
Joyce	2	Normal Load
Ramesh	1	Light Load
James	1	Light Load
Jennifer	2	Normal Load
Ahmed	2	Normal Load
Alicia	2	Normal Load

Solution for the previous query

SELECT

e.Fname, COUNT(w.Pno) AS Project_Count,

CASE

WHEN COUNT(w.Pno) >= 3 THEN 'Overloaded'

WHEN COUNT(w.Pno) = 2 THEN 'Normal Load'

ELSE 'Light Load'

END AS Workload_Status

FROM employee e

LEFT JOIN works_on w ON e.Ssn = w.Essn

GROUP BY e.Fname;

The same query using a CTE to show how CASE can be combined with CTE for better readability

```
WITH Project_Counts AS (  
    SELECT e.Fname, COUNT(w.Pno) AS Proj_Count  
    FROM employee e LEFT JOIN works_on w ON e.Ssn = w.Essn  
    GROUP BY e.Fname  
)  
SELECT Fname, Proj_Count,  
    CASE  
        WHEN Proj_Count >= 3 THEN 'Overloaded'  
        WHEN Proj_Count = 2 THEN 'Normal Load'  
        ELSE 'Light Load'  
    END AS Workload_Status  
FROM Project_Counts;
```



THANK YOU

Mahitha G

Department of Computer Science and Engineering

mahithag@pes.edu



Database Management Systems

Window Functions

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

What are window functions?

In MySQL, a window function performs a calculation across a set of rows that are related to the current row, without collapsing rows into a single output (unlike GROUP BY).

Window functions let you do calculations across rows, but instead of grouping them like SUM() or AVG() normally does, they let you keep every row and just add extra calculated values.

How are they different from aggregate functions?

While aggregate functions collapse rows into one result, window functions add new values without collapsing rows.

Database Management Systems

Windows Functions in SQL - Example

Sample schema - Departments and Employees table

```
[mysql> select * from departments;
```

dept_id	dept_name
1	HR
2	Sales
3	Engineering

3 rows in set (0.01 sec)

```
[mysql> select * from employees;
```

emp_id	emp_name	department	salary	hire_date
101	Alice	Sales	5000	2020-01-15
102	Bob	Sales	7000	2019-03-12
103	Charlie	Engineering	9000	2018-06-01
104	David	Engineering	8000	2021-07-19
105	Eva	HR	6000	2020-09-30
106	Frank	Sales	7500	2022-02-11

6 rows in set (0.00 sec)

Database Management Systems

Windows Functions in SQL - Example

```
mysql> SELECT department, AVG(salary)
-> FROM employees
-> GROUP BY department;
```

department	AVG(salary)
Sales	6500.0000
Engineering	8500.0000
HR	6000.0000

3 rows in set (0.00 sec)

On executing the query shown using aggregate function AVG we get a single row for each department.

```
mysql> SELECT
-> emp_name, department, salary,
-> AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
-> FROM employees;
```

emp_name	department	salary	dept_avg_salary
Charlie	Engineering	9000	8500.0000
David	Engineering	8000	8500.0000
Eva	HR	6000	6000.0000
Alice	Sales	5000	6500.0000
Bob	Sales	7000	6500.0000
Frank	Sales	7500	6500.0000

6 rows in set (0.00 sec)

Using window functions, all rows in the employee table are maintained along with average salary of the department they belong to.

Why are window functions required?

Window functions let you analyze data across rows without losing individual rows.

They're required when you want both:

1. Detailed row-level data (every employee, every order, etc.)
2. Aggregates or comparisons across groups (totals, averages, rankings, etc.)

Traditional GROUP BY only gives you summary rows, but window functions give you summary + detail in the same query.

Common use cases of window functions include Ranking & Row Numbers, Running Totals & Cumulative Sums, Moving Averages (Trends), Comparisons to Group Values, Percentiles & Distribution, Lag & Lead (Previous/Next Row Access) etc.

Database Management Systems

Windows Functions in SQL - Types



Types of Window Functions in SQL

- **Aggregate functions with OVER()**
SUM(), AVG(), MIN(), MAX(), COUNT()
→ applied over a partition/window instead of collapsing into one row.
- **Ranking functions**
Some of the ranking functions are ROW_NUMBER() , RANK(), DENSE_RANK()
- **Value functions**
Some of the functions are LEAD(), LAG(), FIRST_VALUE() LAST_VALUE()

Syntax

```
SELECT column_name1, column_name2....  
window_function(column_name2)  
OVER([PARTITION BY column_name1]) AS new_column  
FROM table_name;
```

Key Terms

window_function= any aggregate or ranking function

column_name1 and column_name2 = columns to be selected

column_name2= column on which window function is to be applied

new_column= Name of new column

table_name= Name of table

Database Management Systems

Windows Functions in SQL - Example

Let's now try to understand what this query does in detail:

```
SELECT name, department, salary,AVG(salary)  
OVER (PARTITION BY department)  
AS dept_avg_salary  
FROM employees;
```

1. OVER clause tells SQL to not collapse rows like GROUP BY and instead, calculate an aggregate in a 'window' while keeping all rows visible.
2. PARTITION BY department creates separate "mini-groups" (windows) for each department i.e one window for Sales,one window for Engineering,one window for HR. Within each window, calculations are done.
3. AVG(salary)
Inside each department, SQL calculates the average salary.

1. Aggregate Window Function

Aggregate window functions calculate aggregates over a window of rows while retaining individual rows.

Common aggregate functions include:

SUM(): Sums values within a window.

AVG(): Calculates the average value within a window.

COUNT(): Counts the rows within a window.

MAX(): Returns the maximum value in the window.

MIN(): Returns the minimum value in the window.

Database Management Systems

Aggregate Windows Functions in SQL

Some other examples:

1. To find Maximum salary in each department:

```
SELECT emp_ID, emp_name, Department, Salary, MAX(Salary)
OVER (PARTITION BY Department) AS MaxDeptSalary
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary, MAX(Salary)
[    -> OVER (PARTITION BY Department) AS MaxDeptSalary FROM Employees;
+-----+-----+-----+-----+-----+
| Emp_ID | emp_name | Department | Salary | MaxDeptSalary |
+-----+-----+-----+-----+-----+
| 103 | Charlie | Engineering | 9000 | 9000 |
| 104 | David | Engineering | 8000 | 9000 |
| 105 | Eva | HR | 6000 | 6000 |
| 101 | Alice | Sales | 5000 | 7500 |
| 102 | Bob | Sales | 7000 | 7500 |
| 106 | Frank | Sales | 7500 | 7500 |
+-----+-----+-----+-----+-----+
6 rows in set (0.04 sec)
```

Database Management Systems

Aggregate Windows Functions in SQL

2. Count of employees in each department:

```
SELECT Emp_ID, emp_name, Department, Salary, COUNT(*)  
OVER (PARTITION BY Department) AS DeptEmployeeCount  
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary,  
-> COUNT(*) OVER (PARTITION BY Department) AS DeptEmployeeCount  
-> FROM Employees;  
+-----+-----+-----+-----+-----+  
| Emp_ID | emp_name | Department | Salary | DeptEmployeeCount |  
+-----+-----+-----+-----+-----+  
| 103 | Charlie | Engineering | 9000 | 2 |  
| 104 | David | Engineering | 8000 | 2 |  
| 105 | Eva | HR | 6000 | 1 |  
| 101 | Alice | Sales | 5000 | 3 |  
| 102 | Bob | Sales | 7000 | 3 |  
| 106 | Frank | Sales | 7500 | 3 |  
+-----+-----+-----+-----+-----+  
6 rows in set (0.03 sec)
```

2. Ranking Window Function

Ranking window functions assign a position (rank/number) to each row **within a partition**, usually based on an ORDER BY clause.

Common Ranking Functions:

- 1) **RANK()** : Assigns ranks to rows with the same value receiving the same rank. If there are ties, the next rank is skipped.
- 2) **DENSE_RANK()** : Similar to RANK(), but ranks are not skipped. If there are ties, the next rank is consecutive.
- 3) **ROW_NUMBER()** : Assigns a unique sequential number to each row (1, 2, 3 ...). No ties, every row has a different number.

Database Management Systems

Ranking Window Functions in SQL



1. RANK() Function

- RANK() is a window function that assigns a rank (1st, 2nd, 3rd, ...) to each row within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank, and the next rank is skipped. For example: if two rows are tied at rank 1, the next row will be rank 3 (not 2) as shown below.
- **Example:** select emp_name, department, salary, RANK() OVER (PARTITION by department ORDER BY salary DESC) AS rank_in_dept FROM Employees;

```
[mysql> SELECT emp_name, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept FROM Employees;
```

emp_name	department	salary	rank_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	1
Alice	Sales	5000	3

```
6 rows in set (0.01 sec)
```

Database Management Systems

Ranking Window Functions in SQL

2. DENSE_RANK() Function

- DENSE_RANK() is a window function that also assigns ranks (1st, 2nd, 3rd, ...) to rows within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank.
- No ranks are skipped, Alice is now numbered 2 instead of 3 when rank was used.
- Example: select emp_name, department, salary, DENSE_RANK() OVER (PARTITION by department ORDER BY salary DESC) AS dense_rank_in_dept FROM Employees;

```
mysql> SELECT emp_name, department, salary,
->         DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rank_in_dept
-> FROM Employees;
```

emp_name	department	salary	dense_rank_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	1
Alice	Sales	5000	2

6 rows in set (0.01 sec)

Database Management Systems

Ranking Window Functions in SQL

3. ROW_NUMBER() Function

- The ROW_NUMBER() function assigns a unique sequential number to each row within a partition. The numbering starts at 1 for each partition.
- Unlike RANK() or DENSE_RANK(), there are no ties → every row gets its own distinct number, even if values are equal.
- Example: select emp_name, department, salary, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as row_num_in_dept from Employees;

```
mysql> SELECT emp_name, department, salary,  
-> ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num_in_dept  
-> FROM Employees;
```

emp_name	department	salary	row_num_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	2
Alice	Sales	5000	3

```
6 rows in set (0.04 sec)
```

1: LAG() Function: This function lets you look at the value of a column from a previous row within the same partition.

- It is very useful for comparing current row values with earlier ones (like salary growth, stock price change, etc.).
- You can specify how many rows back you want to look (default = 1 row).

General syntax:

LAG(column_name, offset, default_value)

OVER (PARTITION BY partition_column

ORDER BY order_column)

column_name is the column you want to peek ahead into.

offset is how many rows forward to look (default = 1).

default_value is optional, what to return if there's no row that far ahead (default = NULL).

Example: SELECT emp_name, department, salary, LAG(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
->      LAG(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary  
-> FROM Employees;
```

emp_name	department	salary	prev_salary
Charlie	Engineering	9000	NULL
David	Engineering	8000	9000
Eva	HR	6000	NULL
Bob	Sales	7000	NULL
Frank	Sales	7000	7000
Alice	Sales	5000	7000

6 rows in set (0.05 sec)

Use Cases of LAG():

- Compare current salary with previous salary in the same department.
- Track daily stock price changes by comparing today vs yesterday.
- Calculate differences, growth %, or trends row by row.

2 LEAD() Function

- The LEAD() function lets you look at the value of a column from a future row within the same partition. It is the opposite of LAG().
- Useful for comparing the current row with the *next one* (e.g., predicting trends, seeing who comes after).

General syntax:

```
LEAD(column_name, offset, default_value)
OVER (PARTITION BY partition_column
ORDER BY order_column)
```

column_name is the column you want to peek ahead into.

offset is how many rows forward to look (default = 1).

default_value is optional, what to return if there's no row that far ahead (default = NULL).

Example: SELECT emp_name,department,salary, LAG(salary,1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
->      LEAD(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS next_salary  
-> FROM Employees;
```

emp_name	department	salary	next_salary
Charlie	Engineering	9000	8000
David	Engineering	8000	NULL
Eva	HR	6000	NULL
Bob	Sales	7000	7000
Frank	Sales	7000	5000
Alice	Sales	5000	NULL

6 rows in set (0.04 sec)

Use Cases of LEAD():

- See what the next employee's salary is in the same department.
- Calculate salary differences ahead of time (LEAD(salary) - salary).
- Compare today's stock price with tomorrow's price (useful in analytics & predictions).
- Identify transitions (e.g., who replaces whom in a sequence).

Practical Use Cases for Window Functions

Window functions are extremely versatile and can be used in a variety of practical scenarios. Below are some examples of how these functions can be applied in real-world data analysis.

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount
1	101	Alice	HR	C001	2023-01-05	Jan	1000
2	101	Alice	HR	C002	2023-02-10	Feb	2000
3	101	Alice	HR	C001	2023-03-15	Mar	1500
4	102	Bob	HR	C003	2023-01-20	Jan	1200
5	102	Bob	HR	C001	2023-02-25	Feb	1800
6	103	Charlie	IT	C004	2023-01-10	Jan	7000
7	103	Charlie	IT	C005	2023-02-18	Feb	7500
8	104	David	IT	C006	2023-01-28	Jan	7200

Practical Use Cases for Window Functions

SQL statements to create table and populate the values:

```
CREATE TABLE Sales (SaleID INT PRIMARY KEY, EmpID INT, EmpName VARCHAR(50),  
Department VARCHAR(50), CustomerID VARCHAR(10), SaleDate DATE, Month VARCHAR(10),  
Amount DECIMAL(10,2));
```

```
INSERT INTO Sales (SaleID, EmpID, EmpName, Department, CustomerID, SaleDate, Month,  
Amount)
```

```
VALUES
```

```
(1, 101, 'Alice', 'HR', 'C001', '2023-01-05', 'Jan', 1000),  
(2, 101, 'Alice', 'HR', 'C002', '2023-02-10', 'Feb', 2000),  
(3, 101, 'Alice', 'HR', 'C001', '2023-03-15', 'Mar', 1500),  
(4, 102, 'Bob', 'HR', 'C003', '2023-01-20', 'Jan', 1200),  
(5, 102, 'Bob', 'HR', 'C001', '2023-02-25', 'Feb', 1800),  
(6, 103, 'Charlie', 'IT', 'C004', '2023-01-10', 'Jan', 7000),  
(7, 103, 'Charlie', 'IT', 'C005', '2023-02-18', 'Feb', 7500),  
(8, 104, 'David', 'IT', 'C006', '2023-01-28', 'Jan', 7200);
```

1. Retrieve top 2 Sales by each Employee and rank them.

```
SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount  
DESC) AS SaleRank FROM Sales ) AS ranked WHERE SaleRank <= 2;
```

```
[mysql> SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount DESC) AS SaleRank FROM Sales ) AS ranked WHERE SaleRank <= 2;
```

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount	SaleRank
2	101	Alice	HR	C002	2023-02-10	Feb	2000	1
3	101	Alice	HR	C001	2023-03-15	Mar	1500	2
5	102	Bob	HR	C001	2023-02-25	Feb	1800	1
4	102	Bob	HR	C003	2023-01-20	Jan	1200	2
7	103	Charlie	IT	C005	2023-02-18	Feb	7500	1
6	103	Charlie	IT	C004	2023-01-10	Jan	7000	2
8	104	David	IT	C006	2023-01-28	Jan	7200	1

```
7 rows in set (0.01 sec)
```


2. Running Total of Sales per Employee

SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;
```

EmpID	EmpName	Month	Amount	RunningTotal
101	Alice	Jan	1000	1000
101	Alice	Feb	2000	3000
101	Alice	Mar	1500	4500
102	Bob	Jan	1200	1200
102	Bob	Feb	1800	3000
103	Charlie	Jan	7000	7000
103	Charlie	Feb	7500	14500
104	David	Jan	7200	7200

```
8 rows in set (0.00 sec)
```

3. First Sale of Each Customer

SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales) AS first_sales WHERE rn = 1;

```
[mysql> SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales ) AS first_sales WHERE rn = 1;
```

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount	rn
1	101	Alice	HR	C001	2023-01-05	Jan	1000	1
2	101	Alice	HR	C002	2023-02-10	Feb	2000	1
4	102	Bob	HR	C003	2023-01-20	Jan	1200	1
6	103	Charlie	IT	C004	2023-01-10	Jan	7000	1
7	103	Charlie	IT	C005	2023-02-18	Feb	7500	1
8	104	David	IT	C006	2023-01-28	Jan	7200	1

```
6 rows in set (0.01 sec)
```

4. Compare Current Sale with Previous Sale (LAG)

SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;
```

EmpID	EmpName	Month	Amount	PrevAmount
101	Alice	Jan	1000	NULL
101	Alice	Feb	2000	1000
101	Alice	Mar	1500	2000
102	Bob	Jan	1200	NULL
102	Bob	Feb	1800	1200
103	Charlie	Jan	7000	NULL
103	Charlie	Feb	7500	7000
104	David	Jan	7200	NULL

```
8 rows in set (0.00 sec)
```

5. Rank Employees by Sale Amount within Department

SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;
```

EmpID	EmpName	Department	Amount	RankInDept
101	Alice	HR	2000	1
102	Bob	HR	1800	2
101	Alice	HR	1500	3
102	Bob	HR	1200	4
101	Alice	HR	1000	5
103	Charlie	IT	7500	1
104	David	IT	7200	2
103	Charlie	IT	7000	3

```
8 rows in set (0.00 sec)
```

- 1) MySQL Window Functions Reference
- 2) SQL Server Window Functions (Microsoft Docs)
- 3) Geeks for Geeks



THANK YOU

Mahitha G

Department of Computer Science and Engineering

mahithag@pes.edu



Database Management Systems

Full Text Search in MySQL

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering



Full-Text Search is a feature in MySQL where you can search for words or phrases in large text-based columns (CHAR, VARCHAR, TEXT) using natural language processing (NLP). Instead of just checking if a string is present, full-text search **looks for relevant matches** and **ranks results based on how well they match the search terms**.

Full-Text Search in MySQL allows you to perform advanced text-based searches on columns with textual data. It is particularly useful for searching large text fields like articles, descriptions, or comments. It is more powerful than **LIKE/REGEXP**, since it ranks results by relevance.

Features -

- **Full-Text Indexes** → much faster than **LIKE**
- **Search Modes** → Natural Language, Boolean, Query Expansion
- **Relevance Ranking** → Scores based on closeness of match
- Handles stopwords, synonyms, and word proximity.

The key feature of Full Text Search is that it uses the **MATCH(column) AGAINST('search terms')** syntax to search.

Suppose you have a table called 'Product' with different columns such as product_id, product_name, description and price. Suppose you want to search for the products that have "smartphone" in their name or description. The SQL query for that would be :

```
SELECT * FROM products  
WHERE MATCH(product_name, description)  
AGAINST('smartphone' IN BOOLEAN MODE);
```

Here,

MATCH function is used to perform full text search on product_name and description column.

AGAINST specifies the search term "smartphone" and in boolean mode.

Feature	LIKE/REGEXP	FULL TEXT SEARCH
Index Usage	Does not use indexes effectively. Slow for large data	Uses special indexes Faster searches
Relevance Ranking	No relevance ranking, just returns matches	Results ranked by relevance
Natural Language Support	Not supported	Supported (Natural Language Mode)
Word Boundary Detection	Not accurate (matches substrings like “cat” in “concatenate”)	Accurate word boundary detection
Boolean Logic	Not supported	Supported (AND, OR, NOT operators)
Phrase Searching	Possible but imprecise	Supported and more accurate

By default or with the **IN NATURAL LANGUAGE MODE** modifier, the **MATCH()** function performs a natural language search for a string against a text collection.

A collection is a set of one or more columns included in a FULLTEXT index. The search string is given as the argument to **AGAINST()**.

For each row in the table, **MATCH()** returns a relevance value; that is, a similarity measure between the search string and the text in that row in the columns named in the **MATCH()** list.

Working :

1. Use columns of type VARCHAR or TEXT
2. Create a **FULLTEXT INDEX** on the column(s)
3. Use MATCH(column1, column2) with AGAINST('word')

Example :

```
CREATE TABLE posts (  
    id INT UNSIGNED AUTO_INCREMENT NOT NULL PRIMARY KEY,  
    title VARCHAR(200),  
    body TEXT  
) ENGINE=InnoDB;
```

```
ALTER TABLE posts  
ADD FULLTEXT(title, body); -- To enable FTS
```

```
INSERT INTO posts (title, body) VALUES  
( 'MySQL Tutorial', 'Learn the basics of database systems'),  
( 'Advanced Database', 'We explore indexing and performance'),  
( 'Cooking Tips', 'Always use fresh ingredients'),  
( 'MySQL vs PostgreSQL', 'Comparison of two database systems'),  
( 'Oracle Tips', 'Learn how to use Oracle effectively');
```

Query : Select those rows where either the body or the title have the word 'database'

```
SELECT * FROM posts  
WHERE MATCH (title, body)  
AGAINST ('database' IN NATURAL LANGUAGE MODE);
```

```
[mysql> SELECT * from posts  
[    -> WHERE MATCH (title, body)  
[    -> AGAINST ('database' IN NATURAL LANGUAGE MODE);  
+---+-----+-----+  
| id | title          | body                                     |  
+---+-----+-----+  
|  1 | MySQL Tutorial | Learn the basics of database systems |  
|  2 | Advanced Database | We explore indexing and performance |  
|  4 | MySQL vs PostgreSQL | Comparison of two database systems |  
+---+-----+-----+  
3 rows in set (0.00 sec)
```

Explanation :

- We perform a **full-text search** in **natural language mode** for the word "database".
- It searches the title and body fields for relevance to the term "database".
- Returns rows where the relevance score is **non-zero** (i.e., the word "database" appears and is significant).

Natural Language Mode :

1. Uses natural language processing to rank results based on relevance.
2. Ignores **stopwords** (like "a", "the", etc.) and very short words (less than 4 characters, by default).
3. **Does not require special syntax** (unlike BOOLEAN mode).

Mode	Description
Natural Language Mode	Default mode; ranks relevance
Boolean Mode	Allows operators (+, -, *) for complex queries

In MySQL Full-Text Search, **Relevance** is a numeric score that tells **how closely a row matches the search terms**.

It's a **floating-point number** (0.0, 0.25, 1.2, etc.)

Higher Score = Better Match

Relevance is calculated using:

1. How often the search term appears in the column
2. How rare the term is across all rows.
3. Length of the document (shorter docs = higher relevance if term is present)

Example :

```
SELECT id, title,  
       MATCH(title, body) AGAINST('database') AS relevance  
FROM posts  
WHERE MATCH(title, body) AGAINST('database')  
ORDER BY relevance DESC;
```

What it does:

1. Selects matching rows which match against the word 'database'
2. Shows the relevance score
3. Orders results by relevance descending (highest match first)

Now: What If You Don't Mention Relevance?

SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database');

What happens:

Results are still matched using full-text search.

MySQL will still use the **FULLTEXT** index.

By default, **MySQL** will sort the results by relevance, if and only if:

1. You **don't** use any **ORDER BY**.
2. You're using **MATCH(...) AGAINST(...)** in the **WHERE** clause.
3. The **FULLTEXT** index is being used (not bypassed).
4. It's a **simple query** (no joins).

So yes — even **without ORDER BY** or showing relevance, the rows will still be sorted by **highest relevance, automatically**, under those conditions.

Important Note: If you **add any other ORDER BY clause**, MySQL will stop auto-sorting by relevance.

```
SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database')  
ORDER BY id; -- Will sort by id, not relevance
```

What is BOOLEAN MODE?

BOOLEAN MODE allows you to:

1. Use **operators** (+, -, ", *, etc.) to include, exclude, or group terms.
2. Search for **partial words** using wildcards (*).
3. Write **complex search logic** (AND, OR, NOT-like behavior).
4. Search even if the word is a **stopword** or appears in **every row**.

You must **explicitly write**:

AGAINST ('your_query' IN BOOLEAN MODE)

Syntax Example

```
SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -database' IN BOOLEAN MODE);
```

-- Here, we are including 'mysql' and excluding 'database';

Some Common Boolean Operators

Operator	Meaning	Example
+	Word must be present	+mysql (must contain "mysql")
-	Word must not be present	-database (must not contain "database")
(none)	Word is optional, but affects relevance	database-row with the given word are ranked higher
""	Search exact phrase	"mysql tutorial"
*	Wildcard (suffix only)	data* (matches "data", "database")
()	Group terms	+(mysql database) –must contain mysql or database

Example Queries in Boolean Mode

1. Include one word, exclude another

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -oracle' IN BOOLEAN MODE);

Must contain "mysql", must not contain "oracle"

2. Exact Phrase Match

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('"database systems"' IN BOOLEAN MODE);

Must contain the **exact phrase** 'database systems'

3. Prefix Search with Wildcard

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('data*' IN BOOLEAN MODE);

Matches: "data", "database", "databases", etc.

Only suffix wildcards are supported (e.g., data*), not *data.

4. Grouping Words

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+(mysql database) -oracle' IN BOOLEAN MODE);

Database Management Systems

Boolean Mode - Practical Exercise



Here's a practical exercise on **Full-Text Search in MySQL** using Boolean and Natural Language Modes. It has :

- A sample table 'articles'
- Inserted data
- Practice questions ranging from basic to advanced

CREATE TABLE articles (article_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255), content TEXT, FULLTEXT (title, content));

INSERT INTO articles (title, content) VALUES

('Introduction to Databases', 'Databases are organized collections of data. MySQL is a popular open-source database.'),

('MySQL Optimization Techniques', 'Learn indexing, query plans, and caching to speed up MySQL queries.'),

('Oracle vs MySQL', 'Comparison of Oracle and MySQL databases based on performance and cost.'),

('PostgreSQL Guide', 'PostgreSQL is an advanced open-source RDBMS.'),

('Data Science and SQL', 'SQL plays a key role in data science workflows.'),

('MySQL Tutorial for Beginners', 'This MySQL tutorial covers basic to intermediate topics for new developers.'),

('Cooking for Beginners', 'This guide is about basic cooking techniques and not related to databases.');

Database Management Systems

Boolean Mode - Practical Exercise



1. Retrieve all articles that mention the word **"MySQL"** using natural language mode.
2. Get the relevance score for the term **"MySQL"**.
3. Find articles that **must include "MySQL"** but **must not include "Oracle"**.
4. Find articles that contain **either "tutorial" or "guide"**.
5. Find articles that **must contain "data"** and any word that starts with **"scien"** (like "science", "scientific").

(Refer notes for solutions!)



THANK YOU

Mahitha G

Department of Computer Science and Engineering

mahithag@pes.edu